audita

SMART CONTRACT SECURITY REVIEW · PREPARED FOR

Inflow: Protocol X ($IPX)

A fee-on-transfer liquidity expansion protocol on PulseChain

DATE	COMMIT	REMEDIATION
October 1, 2026	97e898c	5d59775

01 ABOUT

Audita

Audita is a Web3 security company specializing in smart contract security reviews for protocols, decentralized applications, and on-chain infrastructure.

We help teams identify vulnerabilities before they reach production through rigorous manual code review, adversarial security research, architecture analysis, and automated testing.

Our team consists of 50+ senior security researchers with proven experience across DeFi, infrastructure, wallets, trading systems, and complex on-chain applications.

To date, Audita has completed more than 100 security reviews across the Web3 ecosystem, working with teams at different stages ranging from pre-launch protocols to production systems securing significant on-chain value.

Every engagement is approached from both an offensive and defensive perspective: understanding how the system is intended to behave, identifying where those assumptions can break, and determining how an attacker could exploit them.

PREVIOUS WORK	WEBSITE
Audita public reports	audita.io

02 METHODOLOGY

How this review works

Researchers inspect the agreed source scope and audited commit, trace asset flows and privileged actions, challenge trust assumptions, and reproduce credible attack paths with targeted tests where feasible.

Each accepted finding records its impact and recommended remediation. We then review the remediation commit and record the outcome as Fixed, Partly fixed, or Acknowledged for that finding.

Risk classification

Severity reflects both the impact of a successful exploit and the likelihood of reaching it under the stated assumptions.

Likelihood / Impact	High	Medium	Low
High	HIGH	HIGH	MEDIUM
Medium	HIGH	MEDIUM	LOW
Low	MEDIUM	LOW	LOW

Informational findings have no direct security impact and sit outside the matrix.

IMPACT

High	Major asset loss, critical control compromise, or protocol-wide disruption.
Medium	Material but bounded loss or disruption.
Low	Limited loss, disruption, or scope.

LIKELIHOOD

High	Practical under ordinary on-chain conditions.
Medium	Requires specific but plausible conditions.
Low	Requires unusual conditions or disproportionate resources.

DISCLAIMER

A security review is bounded by time, resources, expertise, and the stated scope. It can identify vulnerabilities but cannot guarantee their absence. Resolution statuses describe only the remediation commit reviewed here. Later code, configuration, or integrations may change the risk. Continued testing, monitoring, and independent review are recommended.

03 EXECUTIVE SUMMARY

FINDINGS	FIXED & VERIFIED	ACKNOWLEDGED
32	30	2

SEVERITY DISTRIBUTION

2 High	9 Medium	20 Low	1 Informational

CONTRACTS IN SCOPE

src/IPXToken.sol	src/VaultController.sol
src/ConversionModule.sol	src/LiquidityDeployer.sol
src/ExpansionModule.sol	src/TreasuryController.sol
src/ArbitrageMonitor.sol	src/TWAPOracle.sol
src/InflowConstants.sol	src/lockers/Locker.sol
src/lockers/LpLocker.sol	src/lockers/TeamLocker.sol

TIMELINE · OCTOBER 2026

Audit · 9 days			Remediation · 2 days
KICK-OFF	END OF AUDIT	REMEDIATIONS START	REMEDIATIONS END
September 21, 2026	September 29, 2026	September 29, 2026	September 30, 2026

STARTING COMMIT HASH	97e898c22bdce1f2ae9a0ed7bff3434c36f99c82
FINAL COMMIT HASH	5d59775efe1439526f22f365d92e3c184c928347

04 PROJECT SUMMARY

What Inflow: Protocol X ($IPX) is

Users swap WPLS or USDC for IPX, or sell IPX back into those assets, while a configurable pair-trade tax funds automatically locked protocol-owned liquidity and sequential expansion pools.

Taxed IPX accumulates in the vault. Each eligible sell values surplus, converts a capped share, funds protected seeds, adds liquidity, and locks the LP. Four rails open in order as POL grows, then treasury allocation activates. V2 cumulative prices gate this cycle, and a 30-day migration can move it to a successor controller.

SECURITY POSTURE

BEFORE REMEDIATION 97e898c

TWAP lifecycle and composition

M-01 · M-04 · M-05 · M-06 · M-07 · L-05 · L-09

Empty, stale, misaligned, or unequal pair windows produced prices that did not represent the end-to-end market.

Cycle routing and accounting

H-02 · M-02 · M-03 · M-08 · M-09 · L-02 · L-06 · L-10 · L-14 · L-15 · L-16 · I-01

Partial execution, shared budgets, and stale snapshots could spend cooldowns, misroute capital, or distort telemetry.

Rail seed price and pair lifecycle

H-01 · L-01 · L-04 · L-07 · L-08 · L-12 · L-13

First seeds trusted permissionless pair states, inverse prices, inconsistent floors, and silent reserve gates.

Migration and administrative assumptions

L-03 · L-11 · L-17 · L-18 · L-19 · L-20

Frozen assets, swallowed sync failures, unsupported liquidity, and manually discovered pools broke documented assumptions.

AFTER REMEDIATION 5d59775

30 of 32 findings fixed and verified. The remediation prices first-seed reserve legs from the checked forward route, isolates incomplete rounds from cooldown, hardens TWAP handling, aligns rail budgets with available assets, and refreshes telemetry after state changes. Follow-up patches corrected the treasury share, rail-pair pinning, route fallback, and one-block spike cases found during remediation review.

The two acknowledged findings concern unregistered permissionless pools and user liquidity adds that the token treats as sells. Operators must document those limits, monitor new IPX pools, and avoid presenting direct user liquidity as a supported path.

05 CONTRACTS & ROLES

Core contracts

CONTRACT	ROLE
`src/IPXToken.sol`	Taxes flagged pair trades, triggers cycles on taxed sells, and timelocks controller migration.
`src/VaultController.sol`	Custodies balances, values surplus and POL, and coordinates the maintenance cycle.
`src/ConversionModule.sol`	Converts capped IPX surplus into protected seed, WPLS, and stable balances.
`src/LiquidityDeployer.sol`	Deploys vault capital and reserve IPX, then locks the minted LP.
`src/ExpansionModule.sol`	Opens four rails in sequence and pairs routed assets with reserve IPX.
`src/TWAPOracle.sol`	Stores V2 cumulative-price windows with bounded duration and staleness.
`src/ArbitrageMonitor.sol`	Classifies cross-pool price zones from spot and aligned TWAP observations.

TreasuryController and the 5,555-day and 3,579-day lockers are also in scope.

PRIVILEGED ROLES

Protocol owner multisig. Configures bounded taxes, pricing controls, reserve sources, and migrations.

Reserve wallet. Holds 730M IPX and approves reserve-funded liquidity legs.

Treasury wallet. Receives treasury allocations and changes only after a timelock.

Locker multisigs. Control withdrawals and relocking only after the unlock times.

06 FINDINGS

What we found

ID	TITLE	SEVERITY	STATUS
H-01	First Rail Seed Prices IPX Leg Through Manipulable Inverse TWAP	HIGH	FIXED
H-02	Dust Sell from a Locked Pair Halts POL Deployment	HIGH	FIXED
M-01	TWAP Oracle Registers Empty Pairs Synced in the Same Block	MEDIUM	FIXED
M-02	Pair-to-Pair IPX Hops Pay Only the Buy Tax	MEDIUM	FIXED
M-03	Cycle Decisions Rely on a Manipulable Spot IPX Price	MEDIUM	FIXED
M-04	Early Re-Pins Lock the Arbitrage Zone at Neutral	MEDIUM	FIXED
M-05	Aligned Pin Times Hide Unequal TWAP Window Lengths	MEDIUM	FIXED
M-06	Composed TWAP Routes Multiply Per-Pool Averages	MEDIUM	FIXED
M-07	Closed Price Gap Stays Confirmed for a Full Window	MEDIUM	FIXED
M-08	Stable Legs Sized from the Combined WPLS and Stable Surplus	MEDIUM	FIXED
M-09	Earlier Rail Buys Push Later Rails Under Their TWAP Floor	MEDIUM	FIXED
L-01	Dust LP at a False Ratio Blocks the First Rail Seed	LOW	FIXED
L-02	Declined Stable Legs Still Start the Cooldown	LOW	FIXED
L-03	Frozen Stable Blocks the Migration Meant to Escape It	LOW	FIXED

06 FINDINGS

CONTINUED

ID	TITLE	SEVERITY	STATUS
L-04	Dusted IPX/Rail Pair Blocks Rail Activation	LOW	FIXED
L-05	Stale TWAP Window Falls Back to a Caller-Set Price	LOW	FIXED
L-06	Swap Floor Ignores Fee-on-Transfer Output Tokens	LOW	FIXED
L-07	Rail Swap Floor Built from a Lagging Route TWAP	LOW	FIXED
L-08	Rail Skipped When Only the Best Route Fails the Pair Check	LOW	FIXED
L-09	Raising the TWAP Window Keeps Serving Shorter Averages	LOW	FIXED
L-10	Full Treasury Cut Paid When a Deploy Places Only Dust	LOW	FIXED
L-11	Migration Completes When the Progression Sync Fails	LOW	FIXED
L-12	Reserve Check Silently Skips Fundable Seeds	LOW	FIXED
L-13	Seed Leg Sized at Raw TWAP Under-Fills the Protected Seed	LOW	FIXED
L-14	POL Total Goes Stale After a Seed in the Same Call	LOW	FIXED
L-15	Cycle Event Reports the Pre-Deployment Surplus	LOW	FIXED
L-16	Precision Loss in the IPX Surplus Valuation of D	LOW	FIXED
L-17	User Liquidity Adds Are Taxed as Sells	LOW	ACKNOWLEDGED
L-18	Unregistered IPX Pools Bypass the Tax	LOW	ACKNOWLEDGED
L-19	Unreachable Pair Token Check	LOW	FIXED

06 FINDINGS

CONTINUED

ID	TITLE	SEVERITY	STATUS
L-20	Setters Emit Events Without a Change	LOW	FIXED
I-01	Floor Ratchet Recomputes the IPX Floor Every Cycle	INFORMATIONAL	FIXED

HIGH SEVERITY · 2 FINDINGS

H-01 HIGH FIXED

First Rail Seed Prices IPX Leg Through Manipulable Inverse TWAP

DESCRIPTION

When `ExpansionModule._try` seeds a rail for the first time, `_planSeed` sizes the IPX leg along the forward chain and checks the reserve wallet against that amount:

```
uint256 fairIpx = _usdcToIpx(usdIn);
if (fairIpx == 0) return (plan, false);
plan.ipxLeg = fairIpx;
...
uint256 required = plan.ipxLeg;
```

After the swap, `_try` does not use `plan.ipxLeg`. For an empty or dusted IPX/rail pair it sizes the leg again from the rail tokens received, through `_ipxLegForRail`:

```
if (plan.livePair) return _ipxToMatchRail(tokenAmount, pair);
return _railToIpxTwap(plan.path, tokenAmount);
```

On the direct route, `_railToIpxTwap` prices the rail tokens through the stable/rail pool with the rail token as input:

```
address pair = IUniswapV2Factory(factory).getPair(path[i - 1], path[i]);
amount = _twapOut(pair, path[i], amount);
```

That reads the pool's average of stable-per-rail, which is the inverse of the price the route gates check. The band, the swap floor and the route choice in `_twapHopOut` all consult the pool with the stable as input. A Uniswap V2 pair keeps the two averages apart, so a short spike barely moves one and inflates the other in step with its size. Nothing compares the second number with `plan.ipxLeg`. The only limit is the reserve clamp, which caps the leg at the reserve wallet's balance or allowance. The NatSpec above `_ipxLegForRail` says "The leg is never valued back through the inverse TWAP", which the direct route contradicts.

Anyone can own the direct pool. No stable/rail pool needs to exist on the protocol's factory, and `createPair` is permissionless. The first seed attempt registers both route pools, so both need one window before either can price a seed. Within that window the attacker:

1. Holds PRVX at 200 times its fair price for 4% of the window (about 5 minutes in a 2-hour window) and 5% below fair for the rest.
2. Then, in one transaction, adds about $250,000 on each side to clear the 2% hop cap, calls `update` (it is permissionless), and makes a taxed sell to run the cycle.

The stable-to-PRVX average stays within 1% of fair, so every gate passes and the cheap direct route wins. The PRVX-to-stable average reads 8.6 times fair.

For example, with IPX at $0.0001 and a $5,000 seed, the seed buys $5,000 of PRVX. `_ipxLegForRail` then values it at $42,387 of IPX, and the reserve wallet sends that instead of $5,000. The IPX/PRVX pair opens with IPX at about 1/8.6 of its fair price. The attacker withdraws the deep liquidity, buys the cheap IPX with PRVX, and sells it through the base pools. In the test fixture this nets about $12,200. It nets about $8,800 when B has just passed the $50,000 PRVX step and the base pools are thinner. A 2,000-fold spike over the same 4% makes the leg larger than the reserve, so the clamp sends the whole reserve wallet into the pair.

IMPACT

An attacker can take the value of the reserve IPX that the first seed of each rail places, once for each of the four rails. They need only a pool they own and capital that comes back in the same transaction. The IPX goes into the pair at a fraction of its price and leaves through the protocol's base pools as WPLS and USDC. At launch the reserve wallet holds 730M IPX ($73,000 at launch price), and a large enough spike empties it, which also stops every later reserve-funded leg until the multisig refills it. Later contributions to an active rail are not exposed, because a live pair supplies its own ratio, which is bounded at twice the fair leg and checked against the TWAP chain.

RECOMMENDATION

Size the empty-pair leg from the forward chain that `_planSeed` already checked, scaled to the tokens received:

```
if (plan.livePair) return _ipxToMatchRail(tokenAmount, pair);
return (plan.ipxLeg * tokenAmount) / plan.expectedRail;
```

The swap floor and the band keep the fill between about 0.97 and 1.05 of `expectedRail`, so the leg stays within that range of the fair leg, and the reserve check and the actual pull use the same number. This removes every inverse read from the seed path, on the WPLS route as well. Correct the NatSpec on `_ipxLegForRail` and `_railToIpxTwap` to match.

H-02 HIGH FIXED

Dust Sell from a Locked Pair Halts POL Deployment

DESCRIPTION

Every taxed sell runs the full cycle from inside the seller's transaction:

```
if (isSell && taxAmount > 0 && !inSwap && vaultController != address(0)) {
    try IVaultController(vaultController).checkAndExecute() {}
    catch (bytes memory reason) { emit CycleFailed(reason); }
}
```

The seller therefore picks the state the cycle runs in. A PulseX V2 pair holds its `lock` for the whole of `swap`. On a flash swap it calls `uniswapV2Call` on the recipient before releasing the lock, so any code in that callback runs while the pair is locked. The attacker flash-borrows dust WPLS from the IPX/WPLS pair and repays it in IPX inside the callback. That repayment is a transfer into a pair from a non-exempt address, so the token taxes it as a sell and starts the cycle. The cycle now runs with IPX/WPLS locked and IPX/USDC open.

The cycle has two steps that touch the pairs, and they fail differently. Conversion sells vault IPX leg by leg. `ConversionModule._swap` wraps each router call in its own try/catch, so the WPLS leg reverts against the locked pair and is skipped, while the seed and stable legs fill through IPX/USDC. `execute` returns the IPX sold on the legs that filled, and `checkAndExecute` stamps the cooldown on that partial number:

```
if (ipxConverted > 0) {
    lastExecutionTime = block.timestamp;
}
```

Deployment has no such per-leg isolation. `LiquidityDeployer.deploy` adds the WPLS and stable base legs in one call. The WPLS leg reverts against the locked pair, the revert unwinds the stable leg with it, and `checkAndExecute` catches the whole thing as `DeploymentFailed`. The cycle ends with IPX converted, nothing deployed, and the cooldown set.

Conversion may run once per `COOLDOWN_PERIOD` (30 minutes), and deployment requires a conversion in the same cycle. The stamp therefore blocks both for every honest sell in the next 30 minutes. The attacker's transaction succeeds because every revert along the path is caught. The attacker repeats the sell as the first one after each window opens, and deployment never runs.

Each round costs about 12 IPX (about $0.001 at launch price) plus gas: the attacker borrows and repays dust and pays the 0.5% tax and the AMM fee on it.

IMPACT

Protocol-owned liquidity deployment, rail seeding and progression, and treasury payouts stop for as long as any address spends about $0.001 per 30 minutes. This disables the mechanism the transfer tax exists to fund. The vault keeps converting IPX into stable that it never places.

The hold works in the NEUTRAL zone (lock either base pair) and the STABLE_CHEAP zone (lock IPX/USDC, since the residual routes to IPX/WPLS). It does not work in EMERGENCY, where conversion and deployment both use only the stable pair. With the TWAP oracle wired the zone leaves NEUTRAL only when spot and TWAP agree, so NEUTRAL is the usual state. No funds leave protocol control.

RECOMMENDATION

Stamp the cooldown only on a complete round: gate the `lastExecutionTime` update on every planned conversion leg filling, or on a successful deployment, rather than on `ipxConverted > 0`. Give each base LP leg in `LiquidityDeployer.deploy` its own try/catch, as the rail contributions already have, so one locked pair cannot unwind the whole deployment.

The structural fix is to decouple the cycle from user swaps. A permissionless `poke()` or keeper entry point that runs `checkAndExecute` outside any pair's `swap` removes both the locked-pair vector and the "first sell after the cooldown wins" race.

MEDIUM SEVERITY · 9 FINDINGS

M-01 MEDIUM FIXED

TWAP Oracle Registers Empty Pairs Synced in the Same Block

DESCRIPTION

`TWAPOracle._currentCumulatives` rejects a pair with a zero reserve only when the pair's own timestamp is older than the block:

```
(uint112 r0, uint112 r1, uint32 tsPair) = IUniswapV2Pair(pair).getReserves();
if (tsPair != blockTimestamp) {
    unchecked {
        uint32 timeElapsed = blockTimestamp - tsPair;
        if (r0 == 0 || r1 == 0) revert ReserveZero();
        ...
    }
}
```

Anyone can call `sync()` on an empty pair, which stamps it with the current block. In that block `registerPairFromExpansion` stores the empty pair with a zero baseline. `ExpansionModule._twapHopOut` calls it on every route pool it prices:

```
try oracle.registerPairFromExpansion(pair) {} catch {}
if (!oracle.hasCompletedWindow(pair)) {
```

A Uniswap V2 pair adds nothing to its cumulatives while a reserve is zero. If liquidity arrives late in the window, the first `update` divides a cumulative that covers only the time with liquidity by the full window. Both directions then read low by the same factor, and `hasCompletedWindow` returns true. With liquidity for the last tenth of the window, both averages read ten times low.

In a normal pool the product of the two averages is at least 1. This bug lets an attacker push both below it: they price the pool 1/k off during its short live span, so the stable-to-rail average reads fair while the rail-to-stable average reads k^2 of fair. The route gates (band, swap floor, route choice) read only the first. When the first seed of a rail opens its IPX/rail pair, `ExpansionModule._ipxLegForRail` sizes the IPX leg from the second, by valuing the rail tokens received back through the route pool:

```
if (plan.livePair) return _ipxToMatchRail(tokenAmount, pair);
return _railToIpxTwap(plan.path, tokenAmount);
```

A rail-to-stable average that reads low makes the seed pair the rail tokens with too little IPX.

The attacker creates an empty USDC/rail pool, calls `sync()` and runs the cycle in the same transaction when the rail's routes are first priced. They add liquidity for the last twelfth of the window, and then deepen the pool at a fair price to clear the 2% hop cap. For example, with W = 2 hours and a $4,973 PRVX seed, the rail-to-stable average reads 1/145 of fair. The IPX/PRVX pair opens with 0.0069 times the fair IPX. The attacker spends $308 of IPX and takes $4,472 of the PRVX the seed bought.

IMPACT

An attacker can take most of the first seed of each rail, about $5,000 each and $20,000 across the four rails, with capital that comes back. The rail pair is left thin, which also slows that rail's climb to its $25,000 POL gate. A diluted reading that lands off the band only skips the hop and delays the rail by a window. No base pair is exposed, because the owner registers them with liquidity.

RECOMMENDATION

Check for a zero reserve in `_currentCumulatives` before the timestamp test, so the oracle never registers or pins a pair that is empty, whatever block it was last synced in:

```
(uint112 r0, uint112 r1, uint32 tsPair) = IUniswapV2Pair(pair).getReserves();
if (r0 == 0 || r1 == 0) revert ReserveZero();
if (tsPair != blockTimestamp) {
    ...
}
```

A pair registered with both reserves non-zero adds to its cumulatives for the whole window, so the first pin covers the full span it divides by.

M-02 MEDIUM FIXED

Pair-to-Pair IPX Hops Pay Only the Buy Tax

DESCRIPTION

`IPXToken._update` treats a transfer out of a flagged pair as a buy and a transfer into one as a sell:

```
bool isBuy  = isPair[from] && !isFeeExempt[to] && !isFeeExempt[from];
bool isSell = isPair[to]   && !isFeeExempt[from] && !isFeeExempt[to];
```

A router swap that passes through IPX, such as WPLS to IPX to USDC, sets the first pair's output address to the second pair. The IPX/WPLS pair sends IPX straight to the IPX/USDC pair, so one transfer is both a buy and a sell. The tax then uses one rate:

```
uint256 bps = isBuy ? buyTaxBps : sellTaxBps;
taxAmount = (value * bps) / InflowConstants.BPS_DENOMINATOR;
```

`isBuy` is true, so the hop pays the buy rate and never pays the sell rate. The same trade split into a buy and then a sell pays both.

For example, at the default 0.5% buy and 0.5% sell rates, 100 IPX that pass between the two pools pay 0.5 IPX. As a buy followed by a sell they pay 0.5 + 0.4975 = 0.9975 IPX.

The cycle trigger reads the other flag:

```
if (isSell && taxAmount > 0 && !inSwap && vaultController != address(0)) {
```

The hop counts as a sell, so it runs the cycle based on its buy tax. `setTaxes` says a zero sell tax "stops the protocol cycle entirely" and that "nothing else can trigger it". With the rates at buy 100 and sell 0, a hop through IPX still runs the cycle. With buy 0 and sell 100, the hop pays no tax at all, though its second leg is a sell.

IMPACT

Trades between the IPX pools can route through IPX and pay about half the tax. That includes the arbitrage that keeps the IPX/WPLS and IPX/USDC prices in line, so the vault loses tax on flow it expects to earn from. With a zero sell tax, the owner cannot stop the cycle as documented, because any hop through IPX runs it. No funds leave protocol control. The loss is tax the vault never collects.

RECOMMENDATION

When a transfer is both a buy and a sell, charge both taxes. Take the sell tax on what is left after the buy tax, so the hop pays the same as a buy followed by a sell:

```
uint256 buyTax = isBuy ? (value * buyTaxBps) / InflowConstants.BPS_DENOMINATOR : 0;
if (isSell) sellTax = ((value - buyTax) * sellTaxBps) / InflowConstants.BPS_DENOMINATOR;
taxAmount = buyTax + sellTax;
```

Then run the cycle only when `sellTax > 0`, not `taxAmount > 0`, so a zero sell tax stops the cycle on every path. Plain buys and sells pay the same as before.

M-03 MEDIUM FIXED

Cycle Decisions Rely on a Manipulable Spot IPX Price

DESCRIPTION

The cycle values the vault's spare IPX, WPLS and stable as `D` and sells IPX when `D` clears the stage threshold. The IPX part is priced in two hops, IPX to WPLS off the IPX/WPLS pool and WPLS to USD off the WPLS/stable pool. Only the second hop is checked against the TWAP. The first is read straight from live reserves:

```
uint256 ipxWplsQ = _ipxWplsPriceQ();               // spot, never validated
uint256 ipxStableQ = (ipxWplsQ * wplsStableQ) / WPLS_STABLE_Q_SCALE;
uint256 ipxSurplusUsd = (ipxSurplus * ipxStableQ) / 1e18;
```

Whoever last traded IPX/WPLS sets the rate at which the vault's IPX is valued. A sell is any transfer of IPX into a pair by a non-exempt address, so anyone can run a cycle for a few hundred wei, and the hook runs it before the seller's tokens reach the pair.

The momentum check should catch a sudden rise, since it blocks a sale when the IPX price is more than 5% above the price the previous cycle recorded. But the reference moves on every cycle, whether or not anything sold:

```
lastIpxWplsPriceQ = _ipxWplsPriceQ();
```

One cycle after a pump moves the reference to the pumped price. A second cycle then sees no rise.

Consider stage 4 with a $1,500 threshold. The vault holds 10M spare IPX, $200 of WPLS and $100 of stable. IPX trades at $0.0001, so `D` is $1,300 and no cycle sells. The attacker flash-borrows WPLS and buys IPX/WPLS up to $0.0003. A buy runs no cycle. A dust sell then runs one: `D` reads $3,300, the momentum check sees a 200% rise and blocks the sale, and the cycle records $0.0003 as the reference. A second dust sell reads `D` at $3,300 again, $1,800 over the threshold, which picks the 20% share. Momentum compares $0.0003 against $0.0003 and allows it. The vault sells 20% of its spare IPX. The IPX/WPLS leg skips because spot is more than 10% off that pool's TWAP. The IPX/stable leg, untouched, fills at its fair price. If fundable capital clears the deploy threshold the cycle deploys too. The attacker unwinds and repays in the same transaction, paying the buy and sell tax and round-trip slippage.

Pushing the price down blocks a sale that should have run. If a sale filled in the last 30 minutes, the attacker waits for the first block after the cooldown.

IMPACT

Any account can force or block a sale and a deploy on any block and pick the largest share. The pumped price never reaches a trade: the vault sells at most 0.25% of each pool's IPX, skips any pool more than 10% off its TWAP, sizes deploys from balances, and checks each liquidity add at 5% against TWAP. A sandwich on the sale does not pay either.

What the attacker controls is the decision. The vault sells IPX it was meant to hold, on the attacker's schedule and at the top share, and locks working capital into liquidity before its own rules say it should. With the TWAP in the first hop `D` would have read $1,300 and none of this would run. No funds leave protocol control.

RECOMMENDATION

Price the IPX part from the TWAP oracle, as the WPLS/stable price already is, and fail closed when no window is available:

```
(uint256 ipxWplsQ, bool tracked) = _twapIpxWplsQ();
if (!tracked) return (0, true);
```

Move the momentum reference only when a sale filled, inside the existing `ipxConverted > 0` branch, so free cycles cannot reset it.

M-04 MEDIUM FIXED

Early Re-Pins Lock the Arbitrage Zone at Neutral

DESCRIPTION

The arbitrage monitor composes its TWAP reading from three pair windows, each pinned by a permissionless `TWAPOracle.update`. If the pin times sit more than `windowSeconds / 24` apart the reading is discarded and `getZone` returns neutral:

```
if (_pinSpread(pinnedIpxWpls, pinnedWplsStable, pinnedIpxStable) > _maxWindowSkew(oracle)) {
    return (0, 0, 0);
}
```

The comment assumes the keeper can always re-align the pins. It cannot. `update` only requires `elapsed >= windowSeconds`, so a pin can move later but never earlier. The only way to close a gap is to delay the leading pair's next pin until the others are eligible, and anyone can prevent that by pinning each pair the first block it becomes eligible. The gap then carries forward every window.

One seed event is enough: a single keeper lag, or pairs registered at different times (`addPair` and `registerPairFromExpansion` run per pair). Default two-hour window, all three pinned at `T`. At `T + 2h00` the attacker pins IPX/WPLS. At `T + 2h06` the keeper's batch lands six minutes late. IPX/WPLS reverts with `PeriodNotElapsed`, the other two pin, and the spread is six minutes. At `T + 4h00` the attacker pins IPX/WPLS again, and at `T + 4h06` pins the other two before the keeper can hold them back. Three cheap transactions per window hold the lock. No `update` call or oracle setting shrinks the gap. The owner can remove and re-add the leading pair within five minutes of the others' pin, at the cost of a further window with no TWAP, and the attacker re-locks after the next keeper lag.

IMPACT

Emergency and stable-cheap never fire while the lock holds, even when spot reserves show a 4% emergency. The conversion module keeps the stage split instead of routing the residual to close the gap, and the deployer keeps funding the WPLS pool, rails and treasury through the emergency. The conversion module's own comment says that without the stable-cheap route the gap between the two IPX pools climbs without bound, so the protocol widens it every round and arbitrageurs take the spread. The hold is cheap, silent and unrecoverable by the keeper. No funds leave protocol control.

RECOMMENDATION

Pin the three base pairs as one set with a shared `timestampLast`, so one `update` pins all three and the spread is zero by construction. Make a same-block repeat call a no-op instead of a revert so batching does not break:

```
if (elapsed < windowSeconds) {
    if (elapsed == 0) return;
    revert PeriodNotElapsed(elapsed, windowSeconds);
}
```

Have the cycle call `update` at cycle end, as the oracle's comment already claims it does, or gate `update` on the base pairs to the cycle and a keeper role.

M-05 MEDIUM FIXED

Aligned Pin Times Hide Unequal TWAP Window Lengths

DESCRIPTION

The arbitrage monitor accepts a composed TWAP reading only when the three pin times sit within `windowSeconds / 24` of each other:

```
if (_pinSpread(pinnedIpxWpls, pinnedWplsStable, pinnedIpxStable) > _maxWindowSkew(oracle)) {
    return (0, 0, 0);
}
```

The check assumes windows that end together cover the same span. `TWAPOracle.update` averages each pair from its own last pin to now, and `windowSeconds` is a minimum wait, not a length:

```
elapsed = currentTimestamp - snapshot.timestampLast;
if (elapsed < windowSeconds) revert PeriodNotElapsed(elapsed, windowSeconds);
snapshot.price0AverageQ = (price0Cumulative - snapshot.price0CumulativeLast) / elapsed;
```

A pair that missed one pin gets a two-window average in the same block as pairs that get a one-window average. The snapshot stores only the pin time, so the monitor cannot see the length and the spread check passes at zero.

The `_twapPrices` comment bounds the phantom gap at 1/24 of the window's price motion, which would require 72% price motion to fake a 3% gap. That holds only for equal lengths. With a one-window mismatch the phantom is the difference between the two spans' averages, about half the trend over the extra span, and 7% per window is enough.

A pin is missed whenever the keeper skips one pair, or whenever anyone front-runs a keeper that pins all three in one transaction, as `script/OpenTrading.s.sol` does: pinning one pair the block it matures reverts the whole batch with `PeriodNotElapsed`.

Consider the default two-hour window with all three pairs pinned at `T` and IPX drifting down 7% per window in both pools, with no cross-pool gap at any instant. At `T + 2h` the attacker pins IPX/USDC and the keeper's batch reverts. At `T + 4h` the batch lands. All three pins read `T + 4h`, with zero spread, but IPX/USDC averages two hours and the others average four. Price A from the two long windows averages 93.0 cents. Price B from the short one averages 89.5 cents. The monitor reads a 390 bps gap with IPX/USDC cheap. A 3.1% spot gap in the same direction, about $60 at launch depth, makes `getZone` return stable-cheap. Pinning a different pair, or a rising trend, fakes emergency the same way. The reading stands until the next aligned pin, up to a full window and four cycles.

IMPACT

A false emergency zeroes the WPLS leg in `ConversionModule._planLegs` and `wplsShareUsd` in `LiquidityDeployer.deploy`: the whole round goes into IPX/USDC, the rails and treasury get nothing that cycle, and the protocol opens a real gap in IPX/USDC for arbitrageurs. A false stable-cheap sends the residual to IPX/WPLS. Repeating the front-run every window against a batching keeper keeps two pairs stale and locks the zone at neutral through a genuine crash. Setup costs one missed or front-run pin. The phantom needs a real trend of about 7% per window and a $60 spot nudge. No funds leave protocol control.

RECOMMENDATION

Store the elapsed length in the snapshot, return it alongside the pin time, and have `_twapPrices` require the three lengths to match within the same tolerance it applies to the pin times:

```
struct Snapshot {
    ...
    uint32 lastUpdateAt;
    uint32 lastWindowLength; // elapsed used by the last update()
}
```

Alternatively, pin the three base pairs as one set with a shared `timestampLast`, so the lengths agree by construction. That is the same remedy as the pin-drift finding and closes both.

M-06 MEDIUM FIXED

Composed TWAP Routes Multiply Per-Pool Averages

DESCRIPTION

The protocol prices IPX in dollars through two pools and rail tokens through three. Wherever it needs the TWAP of such a route it consults each pool's own window average and multiplies. Each pinned value is an arithmetic mean of that pool's price over the window:

```
// UniswapV2Pair._update
price0CumulativeLast += (reserve1 * Q112 / reserve0) * timeElapsed;
// TWAPOracle.update
snapshot.price0AverageQ = (price0Cumulative - snapshot.price0CumulativeLast) / elapsed;
```

Five sites multiply those means:

```
// ArbitrageMonitor._twapPrices: zone confirmation
priceA = Math.mulDiv(qIpxWpls, qWplsStable, TWAP_Q112);
// VaultController._ipxToUsd: POL valuation, rail thresholds, treasury activation
wplsOut = oracle.consult(ipxWplsPair, ipxToken, ipxAmount);
return _wplsToUsd(oracle, wplsOut);
// ExpansionModule._usdcToIpx: opening price of a new IPX/rail pool, pre-swap fair price
wplsAmount = _twapOut(wplsStablePair, stable, usdAmount);
return _twapOut(ipxWplsPair, wpls, wplsAmount);
// ExpansionModule._railValueUsd then _usdcToIpx: rail→WPLS→USDC→WPLS→IPX
fairIpx = _usdcToIpx(_railValueUsd(path, railAmount));
// ExpansionModule._twapRouteOut: minimum output floor for the rail seed swap
amount = _twapHopOut(pair, path[i], path[i + 1], amount, rail);
```

The TWAP of a two-pool route is `avg(a × b)`. The code computes `avg(a) × avg(b)`. Those differ whenever `a` and `b` move inside the window, and on these routes they always move together: when PLS drops, arbitrage raises WPLS-per-IPX and lowers USDC-per-WPLS to hold their product at the dollar price. The code already states the rule at `ExpansionModule._ipxLegForRail`, where the rail leg "is never valued back through the inverse TWAP: each pool's avg(p) × avg(1/p) exceeds 1 whenever the window's price moved". Two different pools compose the same way, and the via-WPLS fair price does the exact round trip that comment avoids.

Window 3600 s, one pin at the end, IPX at $1 throughout, PLS at $1 for 1800 s then $0.50:

POOL	0 TO 1800 S	1800 TO 3600 S
IPX/WPLS	**100 IPX, 100 WPLS**	**100 IPX, 200 WPLS**
WPLS/USDC	**100 WPLS, 100 USDC**	**200 WPLS, 100 USDC**
IPX/USDC	**100 IPX, 100 USDC**	**100 IPX, 100 USDC**

```
WPLS per IPX : (1 × 1800 + 2   × 1800) / 3600 = 1.5
USDC per WPLS: (1 × 1800 + 0.5 × 1800) / 3600 = 0.75
USDC per IPX : (1 × 1800 + 1   × 1800) / 3600 = 1.0
spot: priceA = 1.00   priceB = 1.00   gap 0
twap: priceA = 1.125  priceB = 1.00   gap 12.5%, stable-cheap
```

Spot price A was 1.00 for the whole hour. The composed TWAP reports 1.125. Through the real contracts spot reads 8 bps and the composed TWAP reads 1246 bps. The error scales with the PLS move inside the window: about 40 bps at 20%, 100 bps at 30%, 3% at 45%. No attacker is needed.

IMPACT

The arbitrage monitor's zone gate needs spot and TWAP to agree, and spot is exact, so the phantom shifts every zone threshold by its size and direction. A 20% intra-window PLS move hides a real emergency of 3.0 to 3.4%. A 50% swing hides a 3.5% emergency entirely, and the protocol keeps selling into the pool where IPX is already cheapest. In calm markets a large PLS move plus a $61 nudge fires a zone that misroutes the round.

`polUsd` prices IPX and rail reserves through the same chain, so the $50K to $150K rail thresholds and treasury activation read a biased number. A new rail pool opens at a price off by the bias and arbitrage takes the difference in the first block. The rail seed's output floor and the 5% fair-price band can put the floor above what the pool delivers or fail an honest pool.

The monitor is the only site where the error flips a decision on its own. No funds leave protocol control and the trigger is market motion.

RECOMMENDATION

No pool accrues a composed price, so V2 cumulatives cannot give its exact average and the bias cannot be removed. Bound it: in the monitor confirm a zone when the TWAP agrees on the cheap side at `ARB_NEUTRAL_BPS` rather than `ARB_EMERGENCY_BPS`, keep the window at the one-hour minimum, value rail to IPX directly along rail to WPLS to IPX to drop the USDC round trip, and size the 3% and 5% bands for the PLS volatility the protocol expects. A 50% swing still defeats the direction check, so this is a mitigation rather than a fix.

M-07 MEDIUM FIXED

Closed Price Gap Stays Confirmed for a Full Window

DESCRIPTION

`ArbitrageMonitor.getZone` classifies the spot reserves and, with the oracle wired, returns that zone only when the completed-window TWAP reading gives the same zone:

```
uint8 spotZone = _zoneOf(priceA, priceB, deviationBps);
if (spotZone == InflowConstants.ZONE_NEUTRAL) return InflowConstants.ZONE_NEUTRAL;
if (twapOracle == address(0)) return spotZone;
(uint256 twapA, uint256 twapB, uint256 twapDev) = _twapPrices();
return _zoneOf(twapA, twapB, twapDev) == spotZone ? spotZone : InflowConstants.ZONE_NEUTRAL;
```

The contract comment states why both readings are required: spot reserves cost about $60 to move at launch depth, so "the dislocation has to be held against arbitrage for a full window."

The TWAP reading is the average that `TWAPOracle.update` pinned at the end of the last completed window. It stays in force until the next `update`, a full `windowSeconds` later (two hours by default), or until `maxTwapStaleness` (four hours by default, up to 24 hours) when the keeper is late:

```
// TWAPOracle.update
if (elapsed < windowSeconds) revert PeriodNotElapsed(elapsed, windowSeconds);
snapshot.price0AverageQ = (price0Cumulative - snapshot.price0CumulativeLast) / elapsed;
snapshot.lastUpdateAt = currentTimestamp;
// TWAPOracle.hasCompletedWindow
return block.timestamp - last <= maxTwapStaleness;
```

The two readings therefore cover different periods. The TWAP covers the last completed window and spot covers the current block. Nothing covers the time between the pin and now. Once a window closes with an average gap of 3% or more, the TWAP keeps confirming that zone for the whole next window, whether or not the gap still exists. For that window, spot alone sets the zone.

No faulty reading is needed. The TWAP is the correct average of a real gap, and spot reflects real reserves. The gap can close in any way: arbitrage, the market moving back, or the protocol's own emergency response.

Example with the default two-hour window, where the keeper pins all three pairs at `T` and `T + 2h`:

1. A real 4% gap opens just after `T - 2h`, with IPX/WPLS the cheap side, and holds for most of the window. `getZone` returns `ZONE_NEUTRAL` throughout, because the live TWAP still covers the calm window before it.
2. The pools rebalance just before `T`. At `T` the keeper pins, and the TWAP reads close to 4% on the emergency side. Spot is balanced, so `getZone` returns `ZONE_NEUTRAL`.
3. At any point before `T + 2h`, a holder buys about $60 of IPX from IPX/USDC, which raises its price by about 3.5% at launch depth ($3,600 paired side), and in the same transaction sends 200 wei of IPX to a pair. That sell runs `VaultController.checkAndExecute`, both readings return `ZONE_EMERGENCY`, and the cycle converts and deploys in emergency. A 3.5% move keeps the zone in emergency when `checkAndExecute` reads it again after the conversion. The holder then sells the IPX back.
4. The holder repeats step 3 once per 30-minute cooldown until the next pin.

The stable-cheap direction works the same way.

IMPACT

A forced `ZONE_EMERGENCY` after the gap has closed sends the whole conversion round to IPX/USDC and the whole deploy budget to IPX/stable. The WPLS pool, the rails and the treasury get nothing that cycle. The deploy budget is half of the WPLS and stable working capital, and emergency asks the stable leg for all of it, so `_addLiquidityStable` clamps it to the full stable surplus. One forced cycle moves all spare stable into LP under the 5,555-day lock. `ExpansionModule.contributeToRail` skips while the stable surplus is short, so the rails also go without in the following cycles until conversions refill it. Because the gap has already closed, the routing opens a gap in the other direction, which the monitor's own comment says "widens the gap and is farmed by arbitrage bots". A forced `ZONE_STABLE_CHEAP` sends the whole conversion residual to IPX/WPLS and starves the stable side that the rails and treasury draw on.

In the example above, the protocol never responds to the real gap. The zone fires for that event only after the gap has closed, and only when someone moves spot.

Anyone can do this in every window that follows a window averaging a gap of 3% or more: about four times per two-hour window at the 30-minute cooldown, and about eight times when the keeper runs to the four-hour staleness limit. Each forced cycle costs the round trip on the spot move, about 1.6% in taxes and swap fees on $60, or $1 to $2 at launch depth, rising with pool depth. The trade does not pay for itself. Each round is capped at 25 bps of the pool's IPX reserve, so a sandwich around it earns about 0.5% of the trader's size against 1.58% in taxes and fees, and the emergency response sells into the pool the trader just made dear. The gain is control over routing. No funds leave protocol control.

RECOMMENDATION

Make the confirming reading end at the current block. Compute each base pair's average from its last pin to now from the live cumulatives, which `TWAPOracle._currentCumulatives` already reads, and require that reading to agree with spot as well once it spans a minimum time such as half a window. A spot move held for one block adds no time to it, and a gap that closed after the pin stops confirming. This leaves no zone in the first part of each window. A sliding-window oracle that stores observations and always averages the last `windowSeconds` avoids that delay.

Accept a pin in the monitor only within `windowSeconds` plus a short grace period, so a late keeper cannot stretch the window to `maxTwapStaleness`.

M-08 MEDIUM FIXED

Stable Legs Sized from the Combined WPLS and Stable Surplus

DESCRIPTION

`LiquidityDeployer.deploy` sizes the whole deploy budget from `VaultController.getFundableUsd`, which adds the USD value of the WPLS surplus to the stable surplus:

```
// VaultController.getFundableUsd
uint256 wplsSurplusUsd = (getWplsSurplus() * wplsStableQ) / WPLS_STABLE_Q_SCALE;
return wplsSurplusUsd + getStableSurplus();
```

It then splits that budget by the allocation table. Only the base WPLS leg spends WPLS. The base stable leg, every rail and the treasury spend stable, in a fixed order:

```
uint256 deployUsd = (fundableUsd * InflowConstants.DEPLOY_SPLIT_PCT) / 100;
...
if (wplsShareUsd > 0) wplsPlacedUsd = _addLiquidityWpls(wplsShareUsd);
if (stableShareUsd > 0) stablePlacedUsd = _addLiquidityStable(stableShareUsd);
...
railPlacedUsd += _contributeToRail(expansion, expansion.prvx(), prvxUsd);
railPlacedUsd += _contributeToRail(expansion, expansion.plsx(), plsxUsd);
railPlacedUsd += _contributeToRail(expansion, expansion.inc(), incUsd);
railPlacedUsd += _contributeToRail(expansion, expansion.pHex(), pHexUsd);
```

Each stable leg checks only the live stable surplus. The base stable leg clamps to it, a rail skips in full when the surplus is short of its slice, and the treasury takes what is left:

```
// LiquidityDeployer._addLiquidityStable
if (stableAmount > stableAvail) stableAmount = stableAvail;
// ExpansionModule.contributeToRail
if (stableAvail < usdStable) return 0;
// LiquidityDeployer.deploy
treasuryPaid = treasuryUsd <= stableSurplus ? treasuryUsd : stableSurplus;
```

This works only while the vault holds WPLS and stable in the ratio the table spends them. `ConversionModule` refills the vault at that ratio, 80/20 with no rail open and 64/36 once one is. Nothing checks the ratio at deploy time, and several paths move it:

- In `ZONE_STABLE_CHEAP`, conversion sends the whole residual to WPLS. The `ConversionModule` header expects this zone to recur, since ordinary rounds push IPX/USDC lower.
- In `ZONE_EMERGENCY`, conversion and deployment send everything to the stable side.
- A move in the WPLS price changes the USD value of the WPLS surplus.
- `getWplsSurplus` reads the vault's WPLS balance, so anyone can raise it with a transfer.

Consider an example with a $90 WPLS surplus and a $10 stable surplus. The budget is $50, and the base share of $40 splits into $32 WPLS and $8 stable:

ACTIVE RAILS	STABLE ASKS	RESULT
PRVX	**base $8, PRVX $10**	**Base takes $8. PRVX needs $10, finds $2, and skips.**
PRVX, PLSX	**base $8, PRVX $2, PLSX $8**	**PRVX takes the last $2. PLSX skips.**
All four, with treasury	**base $8, each rail $2, treasury $2**	**PRVX takes the last $2. PLSX, INC and pHEX skip. Treasury gets $0.**

The stable surplus is spent in full, not half, while the WPLS leg takes $32 of $90. The rail left unpaid is always the newest one, which the table gives the largest share. The reverse happens in a stable-heavy vault: the WPLS leg clamps to the whole WPLS surplus and most of the budget goes unplaced.

IMPACT

The next stage opens only when the previous rail's pair holds at least `RAIL_LP_MIN_USD` ($25,000) of POL. A $5,000 rail seed gives the pair about $10,000, so the newest rail must gain the rest through `contributeToRail`. That rail runs last in the stable queue. While the vault is WPLS-heavy it receives nothing, the next rail cannot open, and the treasury stage cannot open. Once every rail is active, the treasury also misses its 4% cut.

The skew fades once its cause stops. Each cycle takes the whole stable surplus but only about a third of the combined surplus from WPLS, and new conversions arrive at 64/36, so the excess WPLS shrinks by about a third per cycle. Recurring `ZONE_STABLE_CHEAP` rounds keep it in place, and a WPLS transfer to the vault recreates it at any time. The transfer costs the sender the full amount, since nothing returns it. No funds leave protocol control.

RECOMMENDATION

Size each asset from its own surplus. Fund the base WPLS leg from half the WPLS surplus, and fund the base stable leg, the rails and the treasury from half the stable surplus:

```
uint256 wplsBudgetUsd = (wplsSurplusUsd * InflowConstants.DEPLOY_SPLIT_PCT) / 100;
uint256 stableBudget = (stableSurplus * InflowConstants.DEPLOY_SPLIT_PCT) / 100;
```

Split `stableBudget` across the stable legs in the table's proportions, so each rail gets its share instead of the first rail in line taking all of it. Cap each leg at its asset's budget rather than at the live surplus. This restores 50% retention for each asset and removes the effect of a WPLS transfer to the vault.

M-09 MEDIUM FIXED

Earlier Rail Buys Push Later Rails Under Their TWAP Floor

DESCRIPTION

`LiquidityDeployer.deploy` funds the rails one after another in the same transaction:

```
railPlacedUsd += _contributeToRail(expansion, expansion.prvx(), prvxUsd);
railPlacedUsd += _contributeToRail(expansion, expansion.plsx(), plsxUsd);
railPlacedUsd += _contributeToRail(expansion, expansion.inc(), incUsd);
railPlacedUsd += _contributeToRail(expansion, expansion.pHex(), pHexUsd);
```

Each `contributeToRail` swaps stable for the rail token. On the route through WPLS, the first hop buys WPLS from the WPLS/stable anchor. `_twapHopOut` prices that hop from the anchor's last completed TWAP window and returns the TWAP-priced amount, which `_swapStableForToken` turns into the swap floor:

```
// ExpansionModule._twapHopOut
return (twapOut * InflowConstants.AMM_FEE_NUM * reserveIn)
    / (InflowConstants.AMM_FEE_DEN * reserveIn + InflowConstants.AMM_FEE_NUM * amountIn);
// ExpansionModule._swapStableForToken
uint256 minOut =
    (expected * (InflowConstants.BPS_DENOMINATOR - lpSlippageBps)) /
InflowConstants.BPS_DENOMINATOR;
```

The window was pinned before the cycle, so the TWAP does not reflect the protocol's own buys earlier in the same call, and nobody else can trade between them. Each rail expects the pre-cycle WPLS price while the pool charges a price the protocol itself pushed up. PLSX absorbs the move from PRVX, INC the move from both, and pHEX the move from all three. Once the combined move passes the 3% `lpSlippageBps` but stays under the 5% band in `_twapHopOut`, the band accepts the anchor, the swap reverts on its floor, and pHEX's slice goes back to the vault. pHEX runs last and has the largest slice, 8% of the budget at stage 4.

At stage 4 the three legacy slices take 12% of the deploy budget. They move the anchor past 3% once they reach about 1.8% of its stable reserve, which is a deploy budget of about 15% of that reserve.

For example, against a 500,000 USDC anchor with a $75,000 deploy budget, the three legacy slices of $3,000 each move the anchor about 3.6%. pHEX then reaches its swap, no band or hop-cap skip fires, and its $6,000 slice gets no liquidity.

IMPACT

Treasury activation requires pHEX POL of at least `RAIL_LP_MIN_USD` ($25,000):

```
&& _railPolAtLeast(pHex, InflowConstants.RAIL_LP_MIN_USD)
```

A $5,000 seed gives the pHEX pair about $10,000, so the rest must come from contributions. While deploy budgets stay large against the anchor, every cycle skips pHEX, its POL stays at the seed, and the progression stalls at stage 4 with every other treasury condition met. No attacker is needed, since any taxed sell runs the cycle. The stall ends once deploy budgets fall below the size threshold. The skipped slices stay in the vault. No funds leave protocol control.

RECOMMENDATION

Account for the protocol's own trades within a cycle. Either price each hop from live reserves and compare spot with the TWAP once, at the start of the cycle before the protocol trades, or scale each TWAP-priced output by the anchor's move since the cycle began, so later rails expect the price the earlier rails left behind.

LOW SEVERITY · 20 FINDINGS

L-01 LOW FIXED

Dust LP at a False Ratio Blocks the First Rail Seed

DESCRIPTION

`ExpansionModule` treats an IPX/rail pair as empty, and seeds it by minting at the TWAP-chain price, only while no LP exists. Once any LP is minted, `_planSeed` treats it as live and checks its spot price against the TWAP chain:

```
plan.livePair = r0 > 0 && r1 > 0 && IERC20(pair).totalSupply() > 0;
...
if (pairRatioIpx > 2 * fairIpx) {
    emit RailPriceOutOfBand(rail, pairRatioIpx, fairIpx);
    return (plan, false);
}
if (!_twapPriceOkForExistingPair(pair, rail, plan.path, plan.expectedRail)) return (plan,
false);
```

`_twapPriceOkForExistingPair` skips when spot differs from the reference by more than 5%. An attacker can create the IPX/rail pair and mint a few dollars of LP at a false ratio. From then on, every cycle that tries the rail's first seed emits `PairTWAPDeviationSkipped` and moves nothing. No arbitrage bot trades a pool worth a few dollars, so the skew holds with no further effort. An attacker who burns all LP down to the locked minimum leaves a pair of a few thousand wei that skips in the same way.

For example, with a $5,000 seed and PRVX at $0.01, the attacker mints $2.50 of LP with PRVX 50% dear. Six windows later the rail still has not seeded, and the stage stays at 0. The attacker's LP is still theirs.

The skip is not permanent. Anyone can trade the pair back to fair, which earns about $0.05, and run a cycle in the same transaction through a taxed sell or `poke()`, and the seed then runs. If the trade and the cycle land in separate transactions, the attacker can skew the pair again in between for about $0.05 a time. If the false ratio was more than twice fair, the pair was never registered with the oracle, so the fix costs one more window. For a pair burned to the minimum, a 40-wei donation and a `sync()` bring it within 0.4% of fair.

IMPACT

For a few dollars an attacker stalls PRVX and, since rails open in order, PLSX, INC, pHEX and the treasury stage, while the protected seed stays idle. The stall lasts until someone sends a trade-and-cycle transaction. The protocol has no path of its own out of it. The README's residual covers a seller skipping a rail "for one window" by moving the IPX/rail pair. This version lasts with no further effort. No funds leave protocol control.

RECOMMENDATION

Treat a live pair whose reserves are both within `DUSTED_PAIR_MAX_BPS` of the planned legs like a dusted pair: price it from the TWAP chain, skip the pair check, send both legs to the pair and call `mint`. On a live pair `mint` credits the smaller side's share and leaves the excess to existing LP pro rata. The dust holder owns about 1% of the pool after the mint, so the protocol loses at most about 1% of the excess its false ratio creates.

L-02 LOW FIXED

Declined Stable Legs Still Start the Cooldown

DESCRIPTION

`ConversionModule.execute` marks a round incomplete only when a leg fails after it was submitted. A leg declined before any swap counts as complete:

```
} else if (seedResult == LegResult.Failed) {
    complete = false;
}
...
if (stableResult == LegResult.Filled) stableExecuted += legs.toStable;
else if (stableResult == LegResult.Failed) complete = false;
```

`_swap` declines a leg when spot strays from the TWAP by more than `maxLegDeviationBps` (10% by default):

```
uint256 deviation = _deviationBps(spotOut, twapOut);
if (deviation > maxLegDeviationBps) {
    emit LegSkipped(tokenOut, amountIn, spotOut, twapOut, deviation);
    return LegResult.Declined;
}
```

`VaultController` starts the cooldown whenever any IPX was converted (`if (ipxConverted > 0) lastExecutionTime = block.timestamp;`).

A seller can therefore choose where a round's IPX goes. They buy IPX out of IPX/USDC, which runs no cycle, until the stable-side quote sits more than 10% off the TWAP. Then they make a small taxed sell into IPX/WPLS. The seed leg and the residual stable leg both decline, the WPLS leg fills, and the 30-minute cooldown starts. The seller then sells the IPX back. The momentum gate reads only IPX/WPLS and the monitor still reads neutral, so nothing else stops the round.

For example, at launch depth (3,600 USDC in IPX/USDC), an 8% push costs the seller 4.19 USDC and 1 IPX. The smallest push that works moves about 4.9% of the stable reserve, which costs about $3 to $4 per window at launch and grows with the pool's depth. About 1% of the round trip is tax paid to the vault.

The README accepts that "a rail skip the seller can cause inside their own transaction still lets the round complete and spend the window". This is the same pattern on the conversion legs, which the README does not name.

IMPACT

While a seller keeps this up each window, the protected seed bucket does not grow, because only a filled seed leg credits it. Rail seeds, stage progression and the treasury stage then stall, and the vault's stable balance stops growing, while every round's IPX goes to WPLS. The seller must win the first cycle of every window against the keeper's `poke()` and other sellers. The declined IPX stays in the vault and the next honest round converts it. No funds leave protocol control.

RECOMMENDATION

Leave `lastExecutionTime` unchanged when a planned seed or stable leg declines, and cap how many WPLS-only rounds can run in one window. Another option is to carry the declined slices into the next round. Either way, add this case to the residual in the README.

Treating a declined leg like a failed one, so the round reverts, would not help: a real price move over 10%, or a TWAP window that is immature or stale, would then halt every cycle.

L-03 LOW FIXED

Frozen Stable Blocks the Migration Meant to Escape It

DESCRIPTION

`IPXToken` describes the controller migration as the path out when the stable fails: "if the underlying stable ever becomes restricted or unusable, the multisig can point the token at a v2 module cluster". The migration runs `VaultController.handoffTo`, which sends every operating balance to the successor, with no way to skip one:

```
if (ipxBal > 0) IERC20(ipxToken).safeTransfer(newVaultController, ipxBal);
if (wplsBal > 0) IERC20(wpls).safeTransfer(newVaultController, wplsBal);
if (stableBal > 0) IERC20(stable).safeTransfer(newVaultController, stableBal);
```

If the stable pauses transfers or blacklists the vault while the vault holds any of it, the stable transfer reverts and the whole migration reverts with it. No other path moves the stable first. `rescueToken` rejects the stable outright, and `withdrawTo` makes the same transfer. The playbook's failure table lists "frozen stable, blacklisted address" with the step "Resolve the token issue, then re-queue", which works only if the freeze ends.

The defect needs a stable that can pause or blacklist. The configured USDC is a bridged token. A successor stable chosen under the playbook may carry those controls.

IMPACT

In the case the migration exists for, a stable that stays frozen, the migration can never run. The IPX, the WPLS and all later tax stay with the old controller cluster, which cannot use the frozen stable either. Only the stable's issuer can end the block. No funds leave protocol control.

RECOMMENDATION

Let the handoff proceed without the stable. Make the stable transfer a low-level call that records a failure instead of reverting, and leave the stranded amount withdrawable by the owner once the freeze ends. Alternatively, add an owner flag on the queued migration that excludes the stable from the handoff.

L-04 LOW FIXED

Dusted IPX/Rail Pair Blocks Rail Activation

DESCRIPTION

Before `ExpansionModule._try` seeds a rail, it looks up the IPX/rail pair and skips it when one reserve is zero, or when both reserves are nonzero but no LP was ever minted:

```
(uint112 reserve0, uint112 reserve1,) = IUniswapV2Pair(ipxRailPair).getReserves();
bool oneSided = (reserve0 == 0) != (reserve1 == 0);
if (oneSided) {
    emit PairOneSidedSkipped(token, ipxRailPair);
    return 0;
}
if (reserve0 > 0 && reserve1 > 0 && IERC20(ipxRailPair).totalSupply() == 0) {
    emit PairPreloadedSkipped(token, ipxRailPair);
    return 0;
}
```

Anyone can put a pair in either state. `createPair` on the factory is permissionless, and one wei of the rail token plus `sync` leaves the pair one-sided with zero supply. Nothing in the pair state changes after that, so the skip repeats every cycle. The deploy script creates only the base pairs, and the rail tokens trade freely, so all four rail pairs can be dusted before launch.

The rails open in order. `checkExpansion` skips a rail until the one before it is active, and treasury activation needs pHEX POL. A dusted PRVX pair therefore holds back PLSX, INC, pHEX and the treasury stage.

The comment above the check says the activation waits for "the multisig to rescue the pair via an owner-driven activation flow". No such function exists. `setRailAddress` closes with `lockRails` and would not help, since the pair is keyed by the rail token. The only way out is for someone to transfer both tokens to the pair and call `mint` directly, since `addLiquidity` reverts on a one-sided pair. The pair then counts as live and must pass `_planSeed`, which registers it with the oracle and waits one TWAP window, two hours by default, before the seed can run. Once any LP is minted, the pair cannot return to either state.

IMPACT

Any account can stall rail expansion and the treasury stage for the gas of `createPair` and one wei of a rail token. Recovery needs the multisig to add liquidity outside the protocol with its own funds, which the protocol does not count as POL, and to wait a TWAP window. No funds leave protocol control.

RECOMMENDATION

Handle a dusted pair inside the protocol. When a pair is one-sided or preloaded and its reserves are below a dust bound relative to the planned legs, transfer both legs at the fair ratio straight to the pair and call `mint`, locking the LP as usual. On a pair with zero supply, `mint` credits `sqrt(amount0 * amount1)` from the balances above reserves, so the dust does not set the price. Keep the skip for pairs whose reserves exceed the bound.

Alternatively, add an owner-callable seed path that survives `lockRails`. In either case, correct the comment so it does not describe a rescue path the code lacks.

L-05 LOW FIXED

Stale TWAP Window Falls Back to a Caller-Set Price

DESCRIPTION

Every part of `D`, the cycle's USD measurement, is priced through one rate, `wplsStableQ`, read from live WPLS/stable reserves. The IPX surplus goes through it too, since IPX is quoted in WPLS first:

```
uint256 ipxStableQ     = (ipxWplsQ * wplsStableQ) / WPLS_STABLE_Q_SCALE;
uint256 ipxSurplusUsd  = (ipxSurplus * ipxStableQ) / 1e18;
uint256 wplsSurplusUsd = (wplsSurplus * wplsStableQ) / WPLS_STABLE_Q_SCALE;
return ipxSurplusUsd + wplsSurplusUsd + stableSurplus;
```

The same rate sizes the deploy through `getFundableUsd`, of which the deployer spends half. `_calculateD` checks that spot against the TWAP oracle and stops the cycle above 20%, but the check switches itself off when the oracle has no fresh window:

```
(uint256 twapPriceQ_, bool tracked) = _twapWplsStableQ();
if (!tracked || twapPriceQ_ == 0) return false;    // no window: report "not manipulated"
```

A window counts as fresh only if the keeper updated it in the last four hours by default. Nothing in the protocol calls the oracle's `update`, so this rests on an outside keeper. When it misses four hours the cycle falls back to comparing spot against the spot the previous cycle recorded, then overwrites that record with the current spot whether the cycle sold, deployed or did nothing:

```
if (twapOracle == address(0) || !_twapTracksAnchor()) {
    if (lastWplsStablePriceQ > 0) {
        uint256 deviation = _absDiff(wplsStableQ, lastWplsStablePriceQ) * 10_000 /
lastWplsStablePriceQ;
        if (deviation > InflowConstants.MANIP_GUARD_BPS) return (0, true);   // 20%
    }
}
lastWplsStablePriceQ = wplsStableQ;    // reference moves every cycle
```

Anyone can run a cycle by sending a few wei of IPX to a pair, so anyone can walk the reference. Move the pool 19%, run a cycle, and the reference is 19% higher. Six steps put the rate at three times honest and the check never trips.

Consider stage 4 with a $1,500 sell threshold and a $3,000 deploy threshold. The WPLS/stable window is stale, while the IPX pairs are fresh. The vault holds IPX worth $1,000, WPLS worth $2,000 and $500 of stable, so `D` is $3,500 and `getFundableUsd` is $2,500. The cooldown ends next block. The attacker buys WPLS with stable until the pool rate is 19% up, which moves no IPX and runs no cycle, then sends a few wei of IPX to a pair. The backup check sees 19% and passes. The cooldown stops any sale. The cycle records the new rate. Five more rounds in the same transaction end at three times honest. Next block one more dust transfer runs a cycle with no deviation: `D` reads $9,500 and the vault sells 20% of its spare IPX into the IPX pools, which pass their own checks. `getFundableUsd` reads $6,500 and deploy runs on $3,250 instead of the honest $1,250. The attacker sells the WPLS back. The position is held for two blocks, so it needs real capital rather than a flash loan.

IMPACT

The pumped rate never reaches a trade, since every sale and liquidity add checks its own pool. It controls what the vault decides. The vault sells IPX it was meant to keep, at the top share, on the attacker's schedule. Its deploy budget is half of a value three times reality, so the stable-paid parts, the stable-side liquidity, the rail contributions and the treasury payment drain the vault's stable to zero instead of leaving half behind. Pushed the other way, the rate stalls every sale and deploy while the keeper is down.

The attack needs a keeper outage of more than four hours on the WPLS/stable pair while the IPX pairs still have a window. No funds leave protocol control.

RECOMMENDATION

Fail closed once trading is open. Trading cannot open without a window on all three pairs, so a stale window after launch means the keeper is down. Skip the cycle instead of dropping to the spot check:

```
if (!_twapTracksAnchor()) return (0, true);
```

Have the cycle call the oracle's `update` for each tracked pair at cycle end, wrapped so an unelapsed window does not revert the cycle. If the backup check must stay for test paths, record `lastWplsStablePriceQ` only when a sale filled.

L-06 LOW FIXED

Swap Floor Ignores Fee-on-Transfer Output Tokens

DESCRIPTION

`ConversionModule._swap` sells IPX for WPLS or the stable through the router's fee-on-transfer variant:

```
try IUniswapV2Router02(router)
    .swapExactTokensForTokensSupportingFeeOnTransferTokens(
        amountIn, minOut, path, recipient, type(uint256).max
    ) {
    return true;
} catch {
    emit LegSkipped(tokenOut, amountIn, spotOut, twapOut, type(uint256).max);
    return false;
}
```

That router function records the recipient's balance before the swap and reverts unless it grew by at least `amountOutMin` afterwards. The check is net of any fee the output token takes on transfer. `minOut` comes from the other side of that fee: `_spotAmountOut` reads `getAmountsOut`, which returns what the pair sends, not what the recipient keeps, and the TWAP floor is derived the same way. The module takes the smaller of the two and cuts it by `minSlippageBps`:

```
uint256 minOut =
    Math.mulDiv(expected, InflowConstants.BPS_DENOMINATOR - minSlippageBps,
InflowConstants.BPS_DENOMINATOR);
```

Nothing subtracts an output-side transfer fee. If the stable or WPLS ever charges one, it comes out of the slippage budget. A 1% fee against the default 5% floor leaves 4% of real tolerance. A fee at or above `minSlippageBps` makes the router revert on every leg, and `setSlippage` has no lower bound.

The failure is silent. The `catch` turns the revert into `LegSkipped` and returns `false`, so `convert` reports zero IPX sold, returns the unspent IPX to the vault, and the cycle carries on. The skip event carries the same `type(uint256).max` sentinel as every other skip.

The IPX input leg is not affected: the token exempts the module from tax and the vault sets `inSwap` for the whole cycle, so the pair receives the full `amountIn`. Today's WPLS and USDC take no fee. The gap goes live only if the migration path chooses a different stable. The playbook allows one and checks only that it has six decimals. `ExpansionModule` already budgets PRVX's 1% fee out of its 3% `lpSlippageBps`. The conversion module carries no such note.

IMPACT

With a fee-on-transfer stable the conversion module either trades on a thinner slippage margin than the owner configured or stops selling IPX altogether without reverting. The vault's IPX surplus then never turns into WPLS or stable, which starves the deploy and expansion steps that spend those balances. No funds leave the protocol and the current token set does not trigger it.

RECOMMENDATION

Either rule the case out or account for it. To rule it out, state in the migration playbook that the stable must not take a fee on transfer and check it when the module is wired. To account for it, measure what the recipient receives and compare that against the floor yourself, the way `ExpansionModule._swapStableForToken` reads the balance delta:

```
uint256 balBefore = IERC20(tokenOut).balanceOf(recipient);
try IUniswapV2Router02(router)
    .swapExactTokensForTokensSupportingFeeOnTransferTokens(
        amountIn, 0, path, recipient, type(uint256).max
    ) {} catch { ... }
uint256 received = IERC20(tokenOut).balanceOf(recipient) - balBefore;
if (received < minOutNetOfFee) revert ...;
```

Give `setSlippage` a lower bound above any expected output fee so the floor cannot reject every fill.

L-07 LOW FIXED

Rail Swap Floor Built from a Lagging Route TWAP

DESCRIPTION

`ExpansionModule._twapHopOut` prices each hop of a rail's swap route from the oracle's last completed window. It checks spot against that TWAP with a 5% band, then returns the TWAP-priced output as the hop's amount:

```
uint256 spotOut = (amountIn * reserveOut) / reserveIn;
uint256 deviation = _deviationBps(spotOut, twapOut);
if (deviation > TWAP_EXPANSION_MAX_DEVIATION_BPS) {
    emit PairTWAPDeviationSkipped(rail, pair, spotOut, twapOut, deviation);
    return 0;
}
return (twapOut * InflowConstants.AMM_FEE_NUM * reserveIn)
    / (InflowConstants.AMM_FEE_DEN * reserveIn + InflowConstants.AMM_FEE_NUM * amountIn);
```

`_swapStableForToken` turns that amount into the swap floor, less `lpSlippageBps` (3%):

```
uint256 minOut =
    (expected * (InflowConstants.BPS_DENOMINATOR - lpSlippageBps)) /
InflowConstants.BPS_DENOMINATOR;
```

The TWAP holds the average of the last window a keeper closed. Nothing in the protocol calls the oracle's `update`. When a route pool's price moves between 3% and 5% against the swap, the band accepts the pool, but the pool cannot pay 97% of the TWAP amount. The router reverts, `_swapStableForToken` returns 0, and the stable goes back to the vault. This repeats every cycle until a keeper closes a window that sat wholly at the new price, up to one full window (2 hours by default) after the move. Rail activation in `_try` and per-cycle contributions in `contributeToRail` share this path.

For example, a 100,000 WPLS buy on a WPLS/PRVX pool holding 5,000,000 WPLS moves the pool about 4% off its TWAP. Every cycle in the rest of that window reaches the seed swap and skips it, and the band never fires. Once a keeper rolls the window, the next cycle opens the rail at the same pool price.

IMPACT

After an ordinary 3% to 5% move in a route pool, rail seeds and contributions skip for up to one TWAP window, although the band treats the price as valid. No one has to act for this to happen. The same revert is what stops a front-run that moves the route by 3% to 5%, and the contract cannot tell the two apart. No funds leave protocol control.

RECOMMENDATION

Make the band and the floor agree. Either build the hop's amount from live reserves and keep the TWAP only as the band check, so any price the band accepts can fill:

```
return (amountIn * InflowConstants.AMM_FEE_NUM * reserveOut)
    / (InflowConstants.AMM_FEE_DEN * reserveIn + InflowConstants.AMM_FEE_NUM * amountIn);
```

or narrow `TWAP_EXPANSION_MAX_DEVIATION_BPS` to `lpSlippageBps`, so a pool the floor would reject skips in pre-flight with `PairTWAPDeviationSkipped`. The first gives up front-run protection between 3% and the 5% band. The second keeps it and makes the skip visible.

L-08 LOW FIXED

Rail Skipped When Only the Best Route Fails the Pair Check

DESCRIPTION

`ExpansionModule._routeQuote` prices the direct stable/rail route and the route through WPLS, keeps the ones whose spot fill clears their floor, and returns the one with the larger output:

```
bool directOk = directOut > 0 && _clearsFloor(amountIn, direct, directOut);
bool hopOk    = hopOut > 0 && _clearsFloor(amountIn, viaWpls, hopOut);
if (directOk && (!hopOk || directOut >= hopOut)) return (direct, directOut);
if (hopOk) return (viaWpls, hopOut);
```

When the IPX/rail pair is live, `_planSeed` checks it against that one route only, and gives up if it fails:

```
if (!_twapPriceOkForExistingPair(pair, rail, plan.path, plan.expectedRail)) return (plan,
false);
```

`_twapPriceOkForExistingPair` builds its reference with `_railToIpxTwap(path, railAmount)`, so the reference follows the chosen route. If the direct pool prices the rail cheaper than the WPLS market by more than the 5% band, it wins the route choice. The IPX/rail pair still tracks the WPLS market, so it then reads more than 5% off the direct route's reference, and the slice is skipped. The WPLS route, which would pass, is never tried.

For example, take a $5,000 PRVX slice and a USDC/PRVX pool with $300,000 of USDC that gives 7% more PRVX than the WPLS route. The direct route quotes 522,236 PRVX against 493,572 through WPLS and wins. The pair check reads 678 bps against the direct route and 19 bps against the WPLS route. The slice is skipped with `PairTWAPDeviationSkipped`, and nothing is placed.

The direct pool is not cheap to hold off price. The 2% hop cap means it must hold at least 50 times the slice. Its own band means it must stay off price for a full window. Each arbitrage that brings it back also ends the skip: one arbitrage of $10,322 takes $210 and lets the next contribution through the WPLS route. A legitimate direct pool that drifts more than 5% from the WPLS route skips slices in the same way, with no attacker.

IMPACT

While the direct pool stays more than 5% off, every contribution to that rail is skipped, and so is a first seed into a pair that is already live. The slices stay in the vault, and the treasury cut shrinks with them. The rail's POL stops growing toward the $25,000 gate, which holds back the next rail and the treasury stage. The skip ends when the pool comes back to price. An attacker gains nothing and loses to arbitrage every window. No funds leave protocol control.

RECOMMENDATION

When the pair is live, choose among the routes that pass both their floor and the pair check, best output first. Have `_routeQuote` return both candidates, and let `_planSeed` try them in order. For the route it takes, reset `plan.path` and `plan.expectedRail`, and rerun the twice-fair band and `pairRatioIpx` before it sets `plan.ipxLeg`.

L-09 LOW FIXED

Raising the TWAP Window Keeps Serving Shorter Averages

DESCRIPTION

`TWAPOracle.setWindowSeconds` changes the window for every pair at once. The snapshots already pinned stay as they are:

```
windowSeconds = seconds_;
if (maxTwapStaleness < seconds_ * 2) {
    uint256 lift = seconds_ * 2;
    if (lift > MAX_TWAP_STALENESS_CAP) lift = MAX_TWAP_STALENESS_CAP;
    emit MaxTwapStalenessSet(maxTwapStaleness, lift);
    maxTwapStaleness = lift;
}
```

Each snapshot holds an average over the `elapsed` span of its last `update`, which was at least the window in force at the time. The snapshot does not store that span, and the readers check only its age against the current staleness bound:

```
function hasCompletedWindow(address pair) external view returns (bool) {
    uint32 last = snapshots[pair].lastUpdateAt;
    if (last == 0) return false;
    return block.timestamp - last <= maxTwapStaleness;
}
```

After a raise, every reader accepts an average over the old, shorter span as a completed window under the new policy. It stays in use until the next `update`, which cannot happen until a full new window has passed since the last pin.

Default two-hour window, four-hour staleness. A pair is pinned at `T` with an average over `[T - 2h, T]`. The owner raises the window to four hours, and the staleness bound lifts to eight. Under the old policy the next pin replaced that average at `T + 2h`. Now `update` reverts with `PeriodNotElapsed` until `T + 4h`, and a late keeper leaves the two-hour average in use until `T + 8h`. In steady state under a four-hour policy, no reader ever sees an average shorter than four hours.

`ArbitrageMonitor` reads the live window for its pin-spread tolerance, so a raise from two to four hours widens the tolerance from 300 to 600 seconds at once, while the pinned spans are still two hours. The `_twapPrices` bound of 1/24 of the window's price motion becomes 1/12 until fresh pins land. The monitor's blindness to unequal span lengths is tracked separately in "Aligned pin times hide unequal TWAP window lengths".

IMPACT

Only the owner can raise the window. For up to one new window after the raise, or two with keeper lag, the conversion, deployment, expansion and manipulation-guard paths price off an average that is only as costly to push as the old window, while the configured policy promises the new one. An attacker gains from this only by holding a pushed price across the old window before the raise lands, a cost the old policy already allowed. The effect is to extend the life of that reading, not to make it cheaper.

RECOMMENDATION

Store the elapsed length in the snapshot and have the readers reject a snapshot whose length is below the current `windowSeconds`:

```
struct Snapshot {
    ...
    uint32 lastUpdateAt;
    uint32 lastWindowLength; // elapsed used by the last update()
}
function hasCompletedWindow(address pair) external view returns (bool) {
    Snapshot storage snapshot = snapshots[pair];
    if (snapshot.lastUpdateAt == 0) return false;
    if (snapshot.lastWindowLength < windowSeconds) return false;
    return block.timestamp - snapshot.lastUpdateAt <= maxTwapStaleness;
}
```

Apply the same check in `consult` and `twapPriceQ`. This is the field the unequal-span finding already proposes, so one change serves both. After a raise, readers fall back to their no-TWAP paths until the next pin. If that pause is not acceptable, document that a raise takes full effect only at each pair's next pin, a full new window after its last one.

L-10 LOW FIXED

Full Treasury Cut Paid When a Deploy Places Only Dust

DESCRIPTION

Once the treasury is active, `LiquidityDeployer.deploy` sets aside 4% of the deploy budget for `treasuryWallet` and pays it whenever the cycle placed any capital at all:

```
uint256 treasuryUsd = (deployUsd * treasuryBps) / InflowConstants.BPS_DENOMINATOR;
...
uint256 placedUsd = wplsPlacedUsd + stablePlacedUsd + railPlacedUsd;
...
if (treasuryUsd > 0 && placedUsd > 0) {
    uint256 stableSurplus = IVaultController(vaultController).getStableSurplus();
    treasuryPaid = treasuryUsd <= stableSurplus ? treasuryUsd : stableSurplus;
```

The comment above this check states its purpose: a reserve wallet that cannot fund any IPX leg must not produce "a cut against nothing". The check tests only whether `placedUsd` is nonzero. The payment is not scaled to what was placed, so a cycle that places $1 pays the same cut as one that places the full budget.

Placement shrinks when the reserve wallet runs low. `_clampIpxToReserve` caps each base leg at the reserve's IPX balance or allowance, and the rails skip through the reserve check in `_planSeed`. The deploy script and README have the reserve approve both modules for `type(uint256).max`, so a drained reserve keeps its allowance. Anyone can then send it a dollar of IPX, an untaxed peer transfer. The next cycle places about $1 through the WPLS leg and pays the full cut.

With $100,000 of fundable capital, the deploy budget is $50,000 and the cut is $2,000. The $49,999 left unplaced stays in the vault, so the next cycle pays about 2% of the fundable capital again, at most once per 30-minute cooldown, for another dollar of IPX.

IMPACT

While the reserve wallet is short of IPX, each deploying cycle moves about 2% of the vault's WPLS and stable working capital to `treasuryWallet` with no liquidity placed against it. The case arises on its own as the reserve runs dry, and any account can hold it open for about $1 per cycle. The stable goes to the treasury wallet, not to the caller, so no funds leave protocol control.

RECOMMENDATION

Scale the payment to the share of the budget that was placed:

```
uint256 plannedUsd = deployUsd - treasuryUsd;
uint256 earnedUsd = placedUsd >= plannedUsd ? treasuryUsd : (treasuryUsd * placedUsd) /
plannedUsd;
treasuryPaid = earnedUsd <= stableSurplus ? earnedUsd : stableSurplus;
```

Alternatively, require `placedUsd` to reach a minimum share of the budget before any treasury payment.

L-11 LOW FIXED

Migration Completes When the Progression Sync Fails

DESCRIPTION

During a migration, `VaultController.handoffTo` merges the predecessor's one-way progression flags into the successor's `ExpansionModule` and `TreasuryController`. Both layers of that merge swallow failures:

```
// VaultController.handoffTo
try successor.receiveProgressionState(expansionModule, treasuryController) {}
catch {
    emit ProgressionHandoffFailed(newVaultController);
}
// VaultController.receiveProgressionState
try IExpansionModule(expansionModule).syncActivationFrom(predecessorExpansion) {}
catch {
    emit ProgressionSyncFailed(expansionModule, predecessorExpansion);
}
```

The migration moves the balances and switches the token's controller either way. Nothing checks the successor's flags afterwards.

A failed sync cannot be repeated. `receiveProgressionState` accepts only the token's current controller, which after the switch is the successor itself, and the successor has no code path that calls it. Both `syncActivationFrom` functions accept only their own controller. The remaining routes are `setModules`, which the playbook closes with `lockModules` before the migration, or a second 30-day migration.

The comment on the `try` says the sync must not strand assets when "a legacy or deliberately minimal successor lacks the hook". The same function already makes hard calls to `receiveFloorState` and `receivePolDepositors`, so every successor must implement custom hooks anyway.

The module constructors copy the flags when the successor is built, so a failed sync loses the flags that advanced during the 30-day delay. It loses all of them if the constructor received `address(0)` as the predecessor. The usual cause is a wiring error, such as a successor module built against a different controller.

IMPACT

The successor reports an earlier stage. The stage sets the floors and the convert and deploy thresholds, so those drop back. A rail whose flag was lost counts as inactive, and on its live pair `_try` spends a second fixed stable seed and more reserve IPX. That capital ends up as locked LP rather than lost. A lost treasury flag stops treasury payouts until the treasury activates again. Only the owner can execute a migration, and the Phase 8 check in the playbook would detect the drop but cannot repair it. No funds leave protocol control.

RECOMMENDATION

Make both sync calls hard calls, as `receiveFloorState` already is, so a failed sync reverts the migration. If the `try` must stay, check after it that the successor's `activeStage` and treasury `isActive` are at least the predecessor's, and revert otherwise.

L-12 LOW FIXED

Reserve Check Silently Skips Fundable Seeds

DESCRIPTION

Before any stable leaves the vault, `ExpansionModule._planSeed` checks that the reserve wallet can cover the IPX side of the add. It pads the planned leg by `TWAP_EXPANSION_MAX_DEVIATION_BPS` (5%) and skips if either the balance or the allowance falls short:

```
uint256 required = (plan.ipxLeg * (InflowConstants.BPS_DENOMINATOR +
TWAP_EXPANSION_MAX_DEVIATION_BPS))
    / InflowConstants.BPS_DENOMINATOR;
if (IERC20(ipxToken).balanceOf(reserveWallet) < required) return (plan, false);
if (IERC20(ipxToken).allowance(reserveWallet, address(this)) < required) return (plan, false);
```

The 5% is there for a fill that beats the plan. The swap has a floor but no ceiling, so it can return more rail tokens than `plan.expectedRail`, and the IPX leg grows with it. With a live pair the leg is linear in the rail amount, since both come from `_ipxToMatchRail` on reserves the swap does not touch:

```
plan.ipxLeg = pairRatioIpx;                          // _ipxToMatchRail(plan.expectedRail,
pair)
...
return _ipxToMatchRail(tokenAmount, pair);           // _ipxLegForRail, after the swap
```

`_twapHopOut` builds `expectedRail` with the router's own fee and impact formula, using the TWAP price where the router uses spot. So the fill over the plan is the product of `spot / twap` across the hops. When spot sits at TWAP, the add needs exactly the planned leg. The 5% headroom covers a worst case, but the check applies it to every call.

A reserve holding between 1.00x and 1.05x of the leg therefore skips seeds it could have funded. Take the $5,000 PRVX seed, with a planned leg worth $5,000 of IPX. `required` is $5,250 of IPX. A reserve holding $5,100 of IPX, fully approved, skips the seed even though an add at the planned price needs only $5,000. The rest of the add already handles a fill that beats the plan: `_try` and `contributeToRail` both clamp the IPX leg to the reserve after the swap and send the spare rail tokens to `reserveWallet`.

The skip leaves no trace. Every other exit in the planning path emits an event: `RailPriceOutOfBand`, `PairTWAPNotMaturedSkipped`, `PairTWAPDeviationSkipped`, `PairRouteHopCapSkipped`. These two lines emit nothing. `_try` returns 0 and `checkExpansion` moves to the next rail. `contributeToRail` returns 0, and `LiquidityDeployer._contributeToRail` emits `RailContributionFailed` only on a revert, so a zero return passes without a record. Operators also cannot read `required` ahead of time: it depends on live TWAP prices, `_planSeed` is internal, and it is not `view`, because `_routeQuote` registers pairs with the oracle.

IMPACT

While the reserve sits in that band, a rail due to open stays closed on every cycle, and nothing on-chain says why. Each rail past PRVX needs the one before it active, so one stalled rail holds back the rails after it and the treasury stage. The same check gates `contributeToRail`, so recurring contributions to live rails stop the same way. With a live pair the leg can reach twice the fair IPX before `RailPriceOutOfBand` fires, so the IPX an operator must hold moves with the pair price and is hard to predict. The stable stays in the vault and the rail opens once the multisig adds IPX or allowance, so no funds are at risk. This is Low.

RECOMMENDATION

Emit an event when the reserve check fails, carrying the figures an operator needs to act:

```
event ReserveInsufficientSkipped(address indexed rail, uint256 required, uint256 balance,
uint256 allowance);
uint256 balance = IERC20(ipxToken).balanceOf(reserveWallet);
uint256 allowance = IERC20(ipxToken).allowance(reserveWallet, address(this));
if (balance < required || allowance < required) {
    emit ReserveInsufficientSkipped(rail, required, balance, allowance);
    return (plan, false);
}
```

Then pick one of two paths for the headroom. Check against `plan.ipxLeg` itself and let the clamp after the swap absorb an overshoot of up to 5%, which sends at most that share of the rail tokens back to `reserveWallet`. Or keep the headroom as a hard rule, document that the reserve must hold at least 1.05x of the leg, and add a view that returns the current `required` for each rail so operators can fund to it.

L-13 LOW FIXED

Seed Leg Sized at Raw TWAP Under-Fills the Protected Seed

DESCRIPTION

Once the protected seed bucket is close to its target, `ConversionModule._planLegs` sizes the seed leg as the exact IPX that should close the gap:

```
uint256 stableShortfall = target - held;
if (stableShortfall <= InflowConstants.SEED_DUST_STABLE) return (0, 0);
shortfallIpx = _ipxForStable(stableShortfall);
```

`_ipxForStable` converts the stable shortfall to IPX with the oracle's raw TWAP ratio:

```
uint256 ipxIn = _consult(pair, stable, stableOut);
return ipxIn == 0 ? type(uint256).max : ipxIn;
```

The ratio carries no AMM fee and no curve impact. The swap that follows does. `_swap` takes the same TWAP quote and cuts it by the fee and the reserve formula before it becomes the output floor:

```
twapOut = Math.mulDiv(
    twapOut,
    InflowConstants.AMM_FEE_NUM * inputReserve,
    InflowConstants.AMM_FEE_DEN * inputReserve + InflowConstants.AMM_FEE_NUM * amountIn
);
```

So the sizing and the fill use two different models of the same trade, and the fill is always the smaller. `_fixedSeedLegs` passes `shortfallIpx` through as the seed leg unchanged, so every round in the fixed-seed path credits less stable than the gap it set out to close. The vault records the real balance delta, not the quote, so the next round measures a new, smaller gap and repeats.

The loss per round is the 0.29% fee plus the curve impact. The seed leg is capped at `capBps` of the stable pool's IPX reserve, 25 bps by default, so impact is about the same size. Starting from a $10,000 shortfall with the seed leg at cap:

ROUND	SHORTFALL BEFORE	STABLE CREDITED	SHORTFALL AFTER
1	**$10,000.00**	**$9,946**	**$54**
2	**$54.00**	**$53.84**	**$0.16**
3	**$0.16**	**$0.16**	**under $0.01**

The sequence ends at round 3 because `_seedSizing` stops funding at or below `SEED_DUST_STABLE` and `ExpansionModule._try` accepts a seed short by the same amount. The bucket is funded, but two rounds later than the IPX on hand allowed. Each extra round waits for the 30-minute `COOLDOWN_PERIOD`, a sell that lifts the surplus over `activeConvertThreshold`, and no bullish-momentum block, so in a quiet market the delay runs to hours or longer.

IMPACT

A rail whose seed the protocol could fund in one round opens two or three cycles late. Stage progression waits on it, and the stage drives both the WPLS split in `conversionWplsPctForStage` and the seed share in `seedAllocationBpsForStage`, so every round in between runs on the earlier stage's parameters. The IPX not sold as seed stays in the round as residual or returns to the vault, and no funds leave protocol control.

RECOMMENDATION

Size the seed leg with the model the swap will fill at. Invert the fee and reserve adjustment `_swap` applies, and round up so the leg lands on or just over the target:

```
function _ipxForStable(uint256 stableOut) internal view returns (uint256) {
    if (twapOracle == address(0)) return type(uint256).max;
    address pair = _pairForOutput(stable);
    if (pair == address(0) || !_hasCompletedWindow(pair)) return type(uint256).max;
    uint256 ipxIn = _consult(pair, stable, stableOut);
    uint256 inputReserve = IERC20(ipxToken).balanceOf(pair);
    if (ipxIn == 0 || ipxIn >= inputReserve) return type(uint256).max;
    // Undo the fee and curve cut `_swap` applies to the TWAP quote.
    return Math.mulDiv(
        ipxIn,
        InflowConstants.AMM_FEE_DEN * inputReserve,
        InflowConstants.AMM_FEE_NUM * (inputReserve - ipxIn),
        Math.Rounding.Ceil
    );
}
```

Any overshoot from rounding or a spot price above TWAP is small and already handled: `VaultController` calls `_releaseProtectedSeedAbove(target)` after each expansion check and returns the excess to working capital.

L-14 LOW FIXED

POL Total Goes Stale After a Seed in the Same Call

DESCRIPTION

`VaultController.checkAndExecute` sums the locked POL once, at step 3, and hands the figure to `ExpansionModule.checkExpansion`:

```
uint256 totalPolUsd = _calculateTotalPolUsd();
...
try IExpansionModule(expansionModule).checkExpansion(totalPolUsd, surplusUsd) returns (uint256
spent) {
```

Inside `checkExpansion` the same figure gates every rail in the loop and the treasury flip after it:

```
for (uint256 railIndex = 0; railIndex < 4; ++railIndex) {
    if (_railActive(railIndex)) continue;
    if (totalPolUsd < _railPolThreshold(railIndex)) continue;
    if (
        railIndex > 0
            && (!_railActive(railIndex - 1)
                || !_railPolAtLeast(_rail(railIndex - 1), InflowConstants.RAIL_LP_MIN_USD))
    ) continue;
    ...
    uint256 spent = _try(_rail(railIndex), railIndex, available);
```

A successful `_try` mints LP and locks it in the same call:

```
if (lpMinted > 0) {
    IERC20(pair).forceApprove(lpLocker, lpMinted);
    ILocker(lpLocker).deposit(pair, lpMinted);
}
```

That deposit raises the locked POL, but nothing reads it again. The later loop iterations, the treasury check and the controller's `CycleExecuted` event all carry the pre-seed figure.

Today the stale figure changes no decision, for two reasons the loop does not state. First, a rail past the first needs the previous rail's own POL at `RAIL_LP_MIN_USD` ($25K), read live through `polUsd`. A seed is about $10K, the $5K stable leg plus its matched IPX, so a rail opened this call cannot clear that floor for the next. Second, `polUsd` values a rail pair at $0 until its IPX/rail TWAP has a completed window (`_tokenAmountUsd` in `VaultController`), and a pair created this call has none. Either fact alone blocks a second rail in the same call, and the treasury flip needs the pHEX floor the same way. So in the call where a rail opens, no later gate can pass, and the stale total reaches only the events.

The events do carry it. Both `CycleExecuted` emit sites report the `totalPolUsd` read at step 3, so in the cycle that opens a rail the event omits the LP that cycle locked. The next cycle's value then jumps by the seed with no cycle accounting for it.

IMPACT

Off-chain readers of `CycleExecuted` see a POL total that is low by one seed in every cycle that opens a rail. No gate is affected now. The safety of the single snapshot rests on two facts the loop neither checks nor records: the seed is smaller than `RAIL_LP_MIN_USD`, and a fresh pair values at zero before its first window. Both would have to change together, for example a lower rail floor alongside a spot fallback in `polUsd`, before a rail could open on a total that no longer matches the chain. Nothing marks either fact as load-bearing, so a later edit can remove one without noticing the other. No funds are at risk, so this is Low.

RECOMMENDATION

Recompute the total after each spend instead of carrying the snapshot. Expose `_calculateTotalPolUsd` as a view on the controller and have `checkExpansion` read it at the top of each loop iteration and before the treasury check. In the controller, emit `CycleExecuted` with a total read after expansion, or after deployment, so the event matches the state at the end of the cycle.

If the team keeps the snapshot, add a comment on the loop naming the two facts that make it safe, and a test that opens a rail and asserts the emitted total against the total read after the cycle.

L-15 LOW FIXED

Cycle Event Reports the Pre-Deployment Surplus

DESCRIPTION

`VaultController.checkAndExecute` last computes `surplusUsd` right after a successful conversion:

```
try IConversionModule(conversionModule).execute(convRate, arbZone) returns (uint256 converted)
{
    ipxConverted = converted;
    (surplusUsd, manipulated) = _calculateSurplusUsd();
```

Deployment runs next and moves capital out of the vault, but nothing reads the surplus again before the cycle ends:

```
try ILiquidityDeployer(liquidityDeployer).deploy(surplusUsd, arbZone) returns (bool didDeploy)
{
    deployed = didDeploy;
}
...
emit CycleExecuted(surplusUsd, totalPolUsd, arbZone, deployed);
```

`LiquidityDeployer.deploy` spends half of `getFundableUsd()`, the vault's WPLS and stable surplus, and every leg that places capital withdraws it from the vault:

- `_addLiquidityWpls` and `_addLiquidityStable` pull their WPLS and stable legs through `_pull`, which calls `withdrawTo` on the vault. Only the unused rest comes back through `_refund`.
- `ExpansionModule.contributeToRail` withdraws its stable slice with `withdrawTo` and keeps it as spent once the swap succeeds.
- The treasury slice sends stable from the vault straight to `treasuryWallet`.

`_surplusUsd` sums the vault's IPX, WPLS and stable surplus, so each of these withdrawals lowers it. IPX legs come from `reserveWallet`, not the vault, so nothing offsets the drop. `deploy` does not use the figure it receives (`surplusUsd; // unused`), so `CycleExecuted` is the only place it goes.

Example: after conversion, the vault holds $10,000 of WPLS and stable above its floors. `deploy` places the $5,000 budget in full. The vault now holds $5,000 of working capital, but `CycleExecuted` reports the $10,000 figure from before the deployment. The next cycle's reading then falls by $5,000, and no cycle event shows where it went.

The other emit site, where the cycle stops at the convert threshold, is correct: no capital moves before it. The event's `totalPolUsd` field has the same gap for the LP that deployment locks, which #14 covers.

IMPACT

Off-chain readers of `CycleExecuted` see a surplus that is too high by the capital deployed, in every cycle that reports `deployed = true`. That is up to half the vault's working capital each time. A dashboard that tracks surplus from this event shows funds the vault no longer holds, and it cannot match the next cycle's lower figure to the deployment that caused it. No contract reads the event, and the tests only check that it fires, not what it carries. No gate or funds depend on the value, so this is Low.

RECOMMENDATION

Recompute the surplus after deployment and emit the figure for the end of the cycle, as the expansion branch already does after it spends protected seed:

```
if (deployed) {
    surplusUsd = _surplusUsd(lastWplsStablePriceQ);
}
emit CycleExecuted(surplusUsd, totalPolUsd, arbZone, deployed);
```

Read `totalPolUsd` again at the same point to close the related gap from #14. If the team wants the event to carry the figure deployment was sized from, document that in the event's NatSpec and add a separate field for the surplus after deployment. Add a test that runs a deploying cycle and checks the emitted surplus against `getD()` read after the cycle.

L-16 LOW FIXED

Precision Loss in the IPX Surplus Valuation of D

DESCRIPTION

`VaultController._D` turns the IPX/WPLS price into a dollar price, then values the IPX surplus with it:

```
// wplsStableQ is 1e30-scaled, ipxWplsQ is 1e18-scaled.
// Their product is 1e48-scaled; dividing by 1e30 gives ipxStableQ
// at 1e18 scale (same scale downstream code expects).
uint256 ipxStableQ = (ipxWplsQ * wplsStableQ) / WPLS_STABLE_Q_SCALE;
...
uint256 ipxSurplusUsd  = (ipxSurplus  * ipxStableQ) / 1e18;
```

`ipxStableQ` is stable-wei per IPX-wei times 1e18. With a 6-decimal stable and an 18-decimal IPX, that is the IPX price in micro-dollars as a whole number. At the launch price of $0.0001 it is 100. The first division drops the fraction, so the IPX leg of D reads up to 1% low at launch. It reads up to 50% low between $0.000001 and $0.000002, and zero below $0.000001. `ArbitrageMonitor` already moved its prices to a 1e30 scale for this reason ("At 1e18 a $0.0001 IPX price is the integer 100"). `_D` kept 1e18.

For example, with the launch vault holding 60M IPX above a 30M floor, plus WPLS and stable:

- At $0.00010099 per IPX, `ipxStableQ` is 100. The IPX leg reads $3,000.00 instead of $3,029.70, and D reads $3,500.00 instead of $3,529.70, 0.84% low.
- At $0.00000099, `ipxStableQ` is 0. With WPLS and stable at their floors, the true D is $29.70, above the $25 stage-0 convert gate, but `getD()` returns 0 and no conversion runs.

IMPACT

D sets the convert gate and the 10%, 15% or 20% conversion rate band. It is also reported in `CycleExecuted` and `getD()`. Deploy sizing and the expansion check do not use it. The rounding only ever lowers D, so near a gate or band boundary a round converts one step later or at a lower rate. After a price collapse below $0.000001 the IPX surplus drops out of D completely. No outside caller can steer the error, and no funds leave protocol control.

RECOMMENDATION

Divide once, at the end:

```
uint256 ipxSurplusUsd = Math.mulDiv(ipxSurplus, ipxWplsQ * wplsStableQ, 1e18 *
WPLS_STABLE_Q_SCALE);
```

The product is about 1e32 at real prices, far from overflow. Remove `ipxStableQ` and correct the comment above it.

L-17 LOW ACKNOWLEDGED

User Liquidity Adds Are Taxed as Sells

DESCRIPTION

A sell is any transfer into a pair from a non-exempt address:

```
bool isSell = isPair[to] && !isFeeExempt[from] && !isFeeExempt[to];
```

The router's `addLiquidity` transfers IPX from the user to the pair, and the router is not exempt. So every user liquidity add pays the 0.5% sell tax and runs the full cycle, between the router quoting the amounts and the pair minting:

```
(amountA, amountB) = _addLiquidity(...);                          // quote against
reserves
TransferHelper.safeTransferFrom(tokenA, msg.sender, pair, amountA);
TransferHelper.safeTransferFrom(tokenB, msg.sender, pair, amountB);   // IPX leg: tax + cycle
liquidity = IUniswapV2Pair(pair).mint(to);                        // credits balance -
reserve
```

With IPX as `tokenA`, the pair receives 0.5% less IPX than quoted and the cycle's swap moves the ratio the quote used. The pair credits the smaller side, so the user's WPLS above that ratio goes to existing holders.

With WPLS or the stable as `tokenA`, the user's paired token is already on the pair when the cycle runs. If the cycle swaps into or deploys into that pair, its reserve sync folds the pending WPLS into reserves. `mint` then sees zero new WPLS and reverts with `INSUFFICIENT_LIQUIDITY_MINTED`.

The fixture works around this: `_growIpxWplsPool` marks the user fee-exempt "so addLiquidity isn't tilted by tax on the IPX leg". Real users get no exemption.

IMPACT

Users adding liquidity pay the sell tax, receive about 0.5% less LP than quoted, and leave the remainder to existing holders. With the paired token first, the add reverts whenever the cycle touches that pair. Every user add is also a free cycle trigger. No funds leave protocol control.

RECOMMENDATION

Document that user liquidity is unsupported, or do not run the tax and cycle while a user's tokens are pending on the pair: skip both on a transfer into a pair whose balance already exceeds its reserves, or fire the cycle only from the pair's swap path.

L-18 LOW ACKNOWLEDGED

Unregistered IPX Pools Bypass the Tax

DESCRIPTION

The token taxes a transfer only when one side of it is flagged as a pair:

```
bool isBuy = isPair[from] && !isFeeExempt[to] && !isFeeExempt[from];
bool isSell = isPair[to] && !isFeeExempt[from] && !isFeeExempt[to];
```

Every other transfer is peer-to-peer and pays nothing. The flag is set from two places only. The owner calls `setPair` by hand, and `ExpansionModule` calls `registerPair` for the rail pairs it seeds itself. The deploy script flags the IPX/WPLS and IPX/stable pairs it creates and nothing else.

Nothing stops anyone else from creating a pool. Once trading is open, any holder can call `createPair` on the PulseX factory with IPX and any token, add liquidity, and trade through it. The same holds for a pair on a different factory or any other contract that swaps IPX. The token has no view of which contracts hold its supply, so the new pool is a plain address to it. Buys from it and sells into it pay no tax, and since only taxed sells call `checkAndExecute`, they never run the cycle either.

The owner can respond with `setPair`, but only after finding the pool. The creator can then deploy the next one, and the next. The factory is permissionless and each pool costs one transaction. The owner ends up chasing a list of addresses that only grows, while the flag on any pool the owner misses stays off.

The price gap does the routing. An untaxed pool that reaches a fraction of the official depth quotes a better net price on every trade, so aggregators and price-sensitive traders move volume there on their own. The 0.5% tax is small, so the pull is weak today, but `setTaxes` allows up to 1% and the gap grows with it.

None of the protocol's accounting reads the unregistered pool. `VaultController.setPairs` is locked to the three base pairs, the TWAP oracle tracks only pairs the owner or the expansion module registered, and `ArbitrageMonitor` follows the same three. So the pool cannot skew `D`, the thresholds or the manipulation guard. It only takes volume away from the venues that feed them.

IMPACT

Volume that moves to an unregistered pool pays no tax into the vault and does not trigger the cycle. The vault earns less, conversion and deployment run less often, and the official pairs lose the trades that were meant to fund them. The owner can flag each pool as it appears but has no way to stop new ones, so the protection is manual and reactive. No funds leave the protocol and the base pairs keep their own guards, so this is Low.

RECOMMENDATION

Accept that the flag cannot cover every venue and document it in the audit brief and README, next to `setPair`, so the owner knows to watch the factory's `PairCreated` events and flag new IPX pools. A keeper can do that: read `allPairsLength` on each supported factory, check whether either token is IPX, and call `setPair` for any match.

If the team wants the token to defend itself, tax the transfer when either side is a contract that is not fee-exempt, or when the recipient reports itself as a V2 pair through `token0` / `token1`. Both raise false positives on wallets and vaults, so they need an allowlist and are a design change rather than a fix.

L-19 LOW FIXED

Unreachable Pair Token Check

DESCRIPTION

Before it seeds or tops up a rail, `ExpansionModule` asks the factory for the IPX/rail pair and then reads the pair back to confirm it holds those two tokens. The same block appears in `_try` and in `contributeToRail`:

```
address ipxRailPair = IUniswapV2Factory(factory).getPair(ipxToken, token);
if (ipxRailPair != address(0)) {
    // Reject if the discovered pair does not
    // actually pair (ipxToken, token). Guards against a stale
    // factory record or a spoofed pair address.
    address token0 = IUniswapV2Pair(ipxRailPair).token0();
    address token1 = IUniswapV2Pair(ipxRailPair).token1();
    bool matches = (token0 == ipxToken && token1 == token) || (token0 == token && token1 ==
ipxToken);
    if (!matches) {
        emit PairTokenMismatchSkipped(token, ipxRailPair);
        return 0;
    }
```

A Uniswap V2 factory, and the PulseX fork of it, cannot return a pair that fails this test. `createPair` sorts the two tokens, deploys the pair with CREATE2, calls `initialize(token0, token1)` on it, and only then writes `getPair[token0][token1]` and `getPair[token1][token0]`. The mapping entry and the pair's own `token0` and `token1` come from the same sorted values in the same transaction. Neither can change afterwards: the mapping has no setter, and the pair rejects `initialize` from any caller but the factory. So the "stale factory record" and "spoofed pair address" the comment names are states the factory has no path to.

The only way the branch fires is if `factory` is not a faithful V2 factory. It is immutable and set once in the constructor, so that takes a deployer who points the module at a malicious or broken factory. That actor also controls the pair code the factory deploys, and can return a pair whose `token0` and `token1` read back correctly while `getReserves` and `mint` lie. The guard adds nothing against that threat either.

The tests agree. `test_14v2_tokenMismatch_skipsAndMovesNoFunds` in `ExpansionPairPreflight.t.sol` and the mismatch case in `ExpansionRefundPaths.t.sol` both reach the branch only by overriding the factory's `getPair` return with `vm.mockCall`. No on-chain state reaches it.

IMPACT

Each seed attempt and each rail top-up pays for two extra external calls, at both call sites, for a check that cannot fail. More important, the comment and the `PairTokenMismatchSkipped` event describe a protection the code does not have. A later reader may trust the guard where it would matter, such as a design that lets the owner replace the factory or pass pair addresses in directly. No funds are at risk, so this is Low.

RECOMMENDATION

Remove the mismatch check, the `PairTokenMismatchSkipped` event and the two tests that mock the factory to reach it. Keep the `token0()` read in `_ipxToMatchRail`, which orients the reserves and is needed.

If the team prefers to keep a guard for a future factory swap, document that intent in the comment instead of claiming a stale or spoofed record is possible today, and reach the branch in tests through a factory stub rather than a cheatcode.

L-20 LOW FIXED

Setters Emit Events Without a Change

DESCRIPTION

Most owner setters write the new value and emit an event without comparing it against the stored one. A call that passes the value already in storage leaves state unchanged but still emits `Updated` or `Set`:

```
function setSlippage(uint256 _bps) external onlyOwner {
    if (_bps > 2000) revert InvalidSlippage(); // hard cap at 20%
    minSlippageBps = _bps;
    emit SlippageUpdated(_bps);
}
```

The one-way locks have the same shape. `lockPairs`, `lockModules` and `lockRails` set the flag and emit on every call, so a locked contract can emit `PairsLocked` any number of times. The codebase already uses the opposite convention in `IPXToken.registerPair`, which returns early on `if (isPair[pair]) return;`, so the two conventions sit side by side.

Affected functions:

- `LiquidityDeployer`: `setReserveWallet`, `setSlippage`, `setTWAPOracle`
- `ConversionModule`: `setSlippage`, `setMaxLegDeviation`, `setConversionCap`, `setTWAPOracle`
- `ArbitrageMonitor`: `setPairs`, `setTWAPOracle`, `lockPairs`
- `IPXToken`: `setVaultController`, `setFeeExempt`, `setPair`, `setTaxes`, `setInSwap`
- `TWAPOracle`: `setWindowSeconds`, `setVaultController`, `setMaxTwapStaleness`
- `VaultController`: `setTWAPOracle`, `setModules`, `setPairs`, `lockModules`, `lockPairs`
- `ExpansionModule`: `setReserveWallet`, `setTWAPOracle`, `setRailAddress`, `setExpansionEnabled`, `lockRails`

`ExpansionModule.setSlippage` is the outlier in the other direction. It emits no event at all, so a slippage change there is invisible off-chain.

IMPACT

Indexers, monitors and dashboards that key on these events record updates that never happened. A setter called twice by mistake, such as a re-broadcast multisig transaction or a re-run script, looks like two changes. Events that carry only the new value, such as `SlippageUpdated(_bps)` and `TWAPOracleSet(_oracle)`, give a consumer no way to tell a real change from a repeat. No funds leave protocol control.

RECOMMENDATION

Return early, or revert, when the new value equals the stored one, and emit only after a real write. Reverting matches the existing input checks and surfaces mistaken repeat calls. Returning early matches `registerPair`. Pick one and apply it to every setter:

```
function setSlippage(uint256 _bps) external onlyOwner {
    if (_bps > 2000) revert InvalidSlippage();
    if (_bps == minSlippageBps) revert NoChange();
    minSlippageBps = _bps;
    emit SlippageUpdated(_bps);
}
```

Revert in the one-way locks when the flag is already set. Where an event carries only the new value, add the previous value, as `MaxLegDeviationUpdated(oldBps, bps)` already does, so consumers can detect repeats without the guard. Add the missing event to `ExpansionModule.setSlippage`.

INFORMATIONAL SEVERITY · 1 FINDING

I-01 INFORMATIONAL FIXED

Floor Ratchet Recomputes the IPX Floor Every Cycle

DESCRIPTION

`_updateVaultState` runs at step 2 of every cycle. It bumps `ipxPeak` when the vault's IPX balance sets a new high, then recomputes the floor from the peak and compares it against the stored floor:

```
if (ipxBal > ipxPeak) ipxPeak = ipxBal;
uint256 newIpxFloor = (ipxPeak * InflowConstants.FLOOR_RATCHET_PCT) / 100;
if (newIpxFloor > ipxFloor) {
    ipxFloor = newIpxFloor;
    emit FloorRatcheted(ipxFloor, wplsFloor(), stableFloor());
}
```

The floor is a pure function of the peak. When the peak does not move, `newIpxFloor` equals the value already stored in `ipxFloor` and the comparison fails. The multiply, divide and storage read run on every cycle for no effect.

IMPACT

A small amount of wasted gas on every cycle where the peak is unchanged, which is most of them. No functional effect.

RECOMMENDATION

Guard the recomputation on the peak having moved, for example by returning early when `ipxBal <= ipxPeak`, or by computing and comparing the floor only inside the branch that updates the peak.